

Scalable Gaussian Processes for Integrated and Overlapping Measurements via Augmented State-Space Models

RYAN A. RUBENZAHN,¹ SOICHIRO HATTORI,^{2,3} SIMO SÄRKKÄ,^{4,5} WILL M. FARR,^{6,1} JACOB K. LUHN,⁷ MEGAN BEDELL,¹
AND DANIEL FOREMAN-MACKEY¹

¹Center for Computational Astrophysics, Flatiron Institute, 162 Fifth Avenue, New York, NY 10010, USA

²Department of Astronomy, Columbia University, 538 West 120th Street, Pupin Hall, New York, NY 10027, USA

³American Museum of Natural History, 200 Central Park West, Manhattan, NY 10024, USA

⁴Department of Electrical Engineering and Automation, Aalto University, 02150 Espoo, Finland

⁵ELLIS Institute Finland, Aalto University, 02150 Espoo, Finland

⁶Department of Physics and Astronomy, Stony Brook University, Stony Brook, NY 11794, USA

⁷Jet Propulsion Lab, Pasadena, CA 91125, USA

ABSTRACT

Astronomical measurements are often integrated over finite exposures, which can obscure latent variability on comparable timescales. Correctly accounting for exposure integration with Gaussian Processes (GPs) in such scenarios is essential but computationally challenging: once exposure times vary or overlap across measurements, the covariance matrix forfeits any quasiseparability, forcing $\mathcal{O}(N^2)$ memory and $\mathcal{O}(N^3)$ runtime costs. Linear Gaussian state-space models (SSMs) are equivalent to GPs and have well-known $\mathcal{O}(N)$ solutions via the Kalman filter and Rauch–Tung–Striebel smoother. In this work, we extend the GP–SSM equivalence to handle integrated measurements while maintaining scalability by augmenting the SSM with an integral state that resets at exposure start times and is observed at exposure end times. This construction yields exactly the same posterior as a fully integrated GP but in $\mathcal{O}(N)$ time on a CPU, and is parallelizable down to $\mathcal{O}(N/T + \log T)$ on a GPU with T parallel workers. We present State-space Models for O(Linear/log) GPs (`smolgp`), an open-source Python/JAX package offering drop-in compatibility with `tinygp` while supporting both standard and exposure-aware GP modeling. Since SSMs provide a framework for representing general GP kernels via their series expansion, `smolgp` also brings scalable performance to many commonly used covariance kernels in astronomy that lack quasiseparability, such as the quasiperiodic kernel. The substantial performance boosts at large N will enable massive multi-instrument cross-comparisons where exposure overlap is ubiquitous, and unlocks the potential for analyses with more complex models and/or higher dimensional datasets.

1. INTRODUCTION

Astronomical measurements are often obtained by integrating a signal over a finite period of time, such as accumulating photoelectrons in a CCD pixel during an exposure. If the signal is time variable, the image readout by the CCD represents the average of that latent signal over the exposure. The timescale associated with the astrophysical phenomenon being probed is usually many times longer than the length of an exposure, and so instantaneous models are effective unbiased estimators of the latent process. However, when the exposure length becomes an appreciable fraction or more of the

timescale of interest, especially for stochastic signals, the effects of exposure integration must be accounted for.

Many astronomical time series of stochastic phenomena are well-modeled by a Gaussian Process (GP; [Rasmussen & Williams 2006](#)); see [Aigrain & Foreman-Mackey \(2023\)](#) for a review. One such example is the case of apparent radial velocity (RV) variability induced by dynamic stellar surfaces, which involve a number of distinct but related processes spanning all timescales from minutes to decades ([Crass et al. 2021](#)). The fastest of these—acoustic (e.g. p -mode) oscillations ([Kjeldsen & Bedding 1995](#); [Huber et al. 2011](#))—are around 5.5 minutes for solar-type stars, which is comparable to the length of a typical exposure. In fact, exposures are often intentionally tuned to the expected oscillation timescale (from scaling relations, e.g. [Brown et al. 1991](#); [Kjeldsen](#)

& Bedding 1995) (or an integer-multiple) in an attempt to mitigate their effect on the measured spectrum (Dumusque, X. et al. 2011; Chaplin et al. 2019).

There is significant utility in combining data-streams from multiple instruments to better resolve subtle signals. In the context of RV (more generally, spectral) variability, Luhn et al. (2026) employ GPs to isolate common astrophysical variability from individual instrumental systematics by jointly modeling the RV time series from multiple EPRV solar feeds. The most constraining information as to whether a signal is shared or not across multiple instruments comes from the times where those instruments were simultaneously observing; as such, overlapping exposures are valuable for accurate signal identification. However, observation details vary from instrument to instrument; the Keck Planet Finder (KPF; Rubenzahl et al. 2023) and NEID (Lin et al. 2022) utilize fast exposures (12 s and 55 s), while EXPRES (Llama et al. 2024) exposes to a fixed signal-to-noise (S/N) and thus has variable exposure times (typically 140–178 s), and HARPS-N (Phillips et al. 2016) takes 300 s exposures to average-out the 5.5 minute p -mode oscillations. Exposure-aware models are therefore critical for unbiased inference from such data. These facilities obtain hundreds of spectra every day; jointly modeling all relevant timescales would involve a data volume on the order of 10^5 observations per year.

Since the naive (dense matrix) GP solution scales computationally as $\mathcal{O}(N^3)$, scalable methods are essential to fully utilize these large datasets, especially if we wish to extend such analyses to the spectral level ($\sim 10^5$ pixels per observation). Consequently, much work has been devoted to scalable GP methods by recasting compatible kernels in terms of low-rank generators. For example, *celerite* (Foreman-Mackey et al. 2017) models utilize a basis set of complex exponentials which produce covariance matrices with semiseparable properties conducive to $\mathcal{O}(N)$ linear algebra operations (Ambikasaran et al. 2015). Most commonly used GP kernels in astrophysics (e.g. half-integer Matérn family, or a stochastically-driven damped harmonic oscillator; SHO) can be represented with *celerite* terms, and as such it has seen widespread adoption. *celerite* models have even been extended to parallel (e.g., multiband) time series (Gordon et al. 2020) as well as close-to-diagonal (S+LEAF; Delisle et al. 2020, 2022) noise models.

The general class of scalable GPs is encapsulated by state-space models (SSMs), which are defined for any GP kernel whose power spectral density (PSD) is a rational function (Hartikainen & Särkkä 2010; Särkkä et al. 2013; Särkkä & Solin 2019). For kernels with PSDs which are the sum of Lorentzians, Kelly et al. (2014)

developed the CARMA model, which similarly achieves $\mathcal{O}(N)$ solutions via a Kalman filter. Quasi-separable kernels also represent a structured subclass of SSMs (Eidelman & Gohberg 1999); this connection powers the efficient $\mathcal{O}(N)$ implementation in *tinygp* (Foreman-Mackey et al. 2024) by using the SSM definition as a generator for quasi-separable matrices (QSMs). More directly though, the SSM definition can be itself be used to solve the GP regression problem in $\mathcal{O}(N)$. As a result, there are a number of packages which implement SSM/GP regression such as *BayesNewton*⁸ (Wilkinson et al. 2023) and *Dynamax*⁹ in Python/JAX, and *TemporalGPs.jl*¹⁰ in Julia. While widely used in other fields outside astronomy, SSMs have seen some use in time domain astronomy for modeling stochastic signals in large survey time series (Kelly et al. 2014), jointly modeling multi-band light curves (Hu & Tak 2020), and detecting gravitational wave signals from pulsar timing (Kimpson et al. 2025).

Unfortunately, integrated measurements complicate these approaches, and no existing implementations handle this case. For the specific case of nonoverlapping exposures and constant exposure times, an exposure-integrated *celerite* kernel function is also a *celerite* kernel (see Appendix A and the `TermConvolution` term in *celerite*). Miller et al. (2022) explored a way to scale approximate integrated kernels to large datasets using inducing points. However, none of these approaches can generate an exact covariance matrix for the general case of variable-length and/or overlapping exposures, due to the fact that the integrated kernel will either depend on the measurement-specific exposure length or will require careful logic to account for overlap. Luhn et al. (2026) developed the methodology for this general case by fully constructing the covariance matrix and solving with dense linear algebra. Though, for our motivating use case of EPRV solar datasets ($N \sim 10^5$), even computing a predictive mean and variance for a multicomponent kernel (i.e. a sum of kernels for each stochastic process of interest) is prohibitively expensive, let alone optimizing the hyperparameters. Thus, we seek a scalable solution without sacrificing model flexibility.

With this motivation, we turn to SSMs (see Särkkä & Solin 2019; Särkkä & Svensson 2023, for a detailed introduction and equivalence to GPs) which are equivalent descriptors of stochastic processes as GPs when their defining stochastic differential equation (SDE) is linear and driving noise is Gaussian. SSMs include all

⁸ <https://github.com/AaltoML/BayesNewton>

⁹ <https://github.com/probml/dynamax>

¹⁰ <https://github.com/JuliaGaussianProcesses/TemporalGPs.jl>

QSM kernels and can be generalized to arbitrary kernels at any desired precision by combining basis kernels (Loper et al. 2021), analogous to *celerite*. As an example, Solin & Särkkä (2014a) show how to approximate the quasiperiodic kernel by constructing an SSM from its Taylor series expansion.

In this work, we show how an SSM augmented with an integral state yields an equivalent solution to the full integrated GP approach of Luhn et al. (2026) but is solved in linear time complexity and is compatible with parallel methods (e.g. Särkkä & García-Fernández 2021; Yaghoobi & Särkkä 2025; Yaghoobi et al. 2025). We introduce SSMs and their equivalence to GPs in Section 2. We then develop an augmented SSM with an integral state to accommodate integrated (and possibly overlapping) observations in Section 3. We implemented the full SSM-GP framework described in this paper in Python/JAX (Bradbury et al. 2018), which we make available in the package State-space Models for O(Linear/log) GPs (smolgp¹¹; Rubenzahl & Hattori 2026). We demonstrate the equivalence and performance boost of this approach compared to traditional GP methods in Section 4. Finally, we summarize the method in Section 5 and conclude with other potential applications and future elaborations of this SSM-GP framework for more complex data or modeling requirements.

2. GPs AS STATE-SPACE MODELS

Let us orient ourselves with the definition of SSMs and their relationship to GPs. First, we will refresh ourselves with the GP problem statement (Section 2.1) before introducing SSMs and their solution (Section 2.2), making the connections between these two complimentary frameworks explicit at each stage. We provide a summary of the components of an SSM and their analogy in the GP framework in Table 1.

2.1. The GP problem statement

A temporal GP defines a probability distribution over functions of time t . For a GP with zero mean and covariance defined by the kernel function $k(t, t')$, these functions $f(t)$ are (Rasmussen & Williams 2006)

$$f(t) \sim \mathcal{GP}(0, k(t, t')). \quad (2.1)$$

Our measurements $\mathbf{y} = \{y_n\}_{n=1:N}$ taken at times $\mathbf{t} = \{t_n\}_{n=1:N}$ are noisy samples of the process,

$$y_n = f(t_n) + \epsilon_n, \quad (2.2)$$

where $\epsilon_n \sim \mathcal{N}(0, \sigma_n^2)$ is our Gaussian measurement noise with variance σ_n^2 . Conditioning the GP on the observed data $(\mathbf{t}, \mathbf{y})$ yields the predictive distribution

$$f(t_* | \mathbf{t}, \mathbf{y}) \sim \mathcal{N}(\boldsymbol{\mu}_{GP}(t_*), \boldsymbol{\Sigma}_{GP}(t_*)). \quad (2.3)$$

with mean ($\boldsymbol{\mu}$) and covariance ($\boldsymbol{\Sigma}$) given by

$$\begin{aligned} \boldsymbol{\mu}_{GP}(t_*) &= \mathbf{K}_*^T (\mathbf{K} + \mathbf{R})^{-1} \mathbf{y}, \\ \boldsymbol{\Sigma}_{GP}(t_*) &= \mathbf{K}_{**} - \mathbf{K}_*^T (\mathbf{K} + \mathbf{R})^{-1} \mathbf{K}_*. \end{aligned} \quad (2.4)$$

Above, $\mathbf{K}$ is the covariance matrix computed from the kernel function for all pairs of observations, i.e. $K_{ij} = k(t_i, t_j)$ for $i, j \in [1, N]$. $\mathbf{K}_*$ denotes the kernel function evaluated between the data and an arbitrary set of M “test” points, i.e. $\mathbf{K}_{ij} = k(t_i, t_{*j})$ for $i \in [1, N]$, $j \in [1, M]$. Likewise, $\mathbf{K}_{**}$ is the same for all pairs of test points. $\mathbf{R}$ is the matrix of measurement covariances; in the usual case of independent but heteroskedastic errors, $\mathbf{R} = \text{diag}(\{\sigma_n^2\}_{n=1:N})$.

The kernel function, which itself usually depends on some number of “hyperparameters” ($\boldsymbol{\theta}$), can have many different functional forms as long as $\mathbf{K}$ is positive definite. We often work with stationary kernels, meaning they are functions only of the time difference $\Delta \equiv |t - t'|$. That is, we have $k(\Delta | \boldsymbol{\theta})$ instead of $k(t, t' | \boldsymbol{\theta})$.

Eq. 2.4 suffers from the curse of dimensionality, scaling computationally as $\mathcal{O}(N^3)$ for naive $N \times N$ matrix inversion and multiplication. Even holding these matrices in memory scales as $\mathcal{O}(N^2)$, which can be 100+ GB for $\gtrsim 10^5$ data points. While QSM methods can reduce both to $\mathcal{O}(N)$ for compatible kernels, we seek an approach compatible with the general case of integrated (with variable exposure time) and sometimes overlapping measurements

2.2. The SSM problem statement

See Solin & Särkkä (2014a) for an introduction to reformulating temporal GP regression into a linear Gaussian SSM, which we summarize here. We also provide a full worked example in Appendix B for an SHO. We note here that we define the subscript n as the index for the data points, i.e. $\mathbf{y} = \{y_n\}_{n=1:N}$, and the subscript k for the discretized states, i.e. $\mathbf{x} = \{x_k\}_{k=1:K}$. In this section, n and k can be used interchangeably as we discretize the model at the data points. However, in Section 3 where we consider integrated measurements, we will discretize the model at the exposure start and end times, and so in that case we will have $K = 2N$ states.

In the state-space formalism, we can think of these functions $f(t)$ as instead solving a d -th order linear SDE driven by a white noise process $w(t)$,

¹¹ <https://github.com/smolgp-dev/smolgp>

Table 1. Summary of state-space model matrices and their relation to GPs

Matrix	Size	Name	Description	Analogy to GP
$\mathbf{F}$	$d \times d$	Feedback matrix	Governs the instantaneous deterministic dynamics of the latent state. The eigenvalues encode how perturbations to that state evolve (e.g. decay).	Encodes the characteristic timescales of the process and the global correlation structure (decay, oscillations, periodicity, etc.).
$\mathbf{L}$	$d \times 1$	Noise effect matrix	Maps the driving white noise to the latent state vector and its derivatives. It is simply $(0, \dots, 1)$ for the cases we consider here (i.e. only the highest derivative term is driven by noise).	How smooth the GP kernel is. If $\mathbf{L}$ has multiple nonzero entries, it is like a sum of GPs with varying smoothness.
$\mathbf{Q}_c$	1×1	Spectral density	Defines the amplitude of the white noise driving the process.	Represents the power of the GP kernel's driving noise (related to the kernel's PSD amplitude).
$\mathbf{P}_\infty$	$d \times d$	Stationary covariance	Prior covariance of the latent state (i.e. before observing any data).	Related to the GP kernel's (and its derivative(s)'s) amplitude.
$\mathbf{H}_k$	$D \times d$	Observation model	Projects the latent k^{th} state to the observed space.	Linear transformation from the latent process to the observed $f(t)$.
$\mathbf{R}_k$	$D \times D$	Observation noise	The variance (and covariances, if $D > 1$) of the measurement at time t_k .	The k^{th} element along the diagonal of the noise matrix $\mathbf{R}$.
$\mathbf{A}_k$	$d \times d$	Transition matrix	Maps the state vector forward by the time step Δ_k .	How the latent GP evolves deterministically over a time-lag Δ_k .
$\mathbf{Q}_k$	$d \times d$	Process noise	Accumulated white noise injected into the state over one time step Δ_k .	How the latent GP evolves stochastically over a time-lag Δ_k .

Note: Recall d is the state ($\mathbf{x}$) dimension and D is the data ($\mathbf{y}_n$) dimension; in this work we consider 1D time series, so $D = 1$. k is the state index which runs from 1 to N for N instantaneous data points; for N integrated measurements, $k \in [1, 2N]$.

$$\begin{aligned} \frac{d\mathbf{x}(t)}{dt} &= \mathbf{F}\mathbf{x}(t) + \mathbf{L}\mathbf{w}(t), \\ f(t) &= \mathbf{H}\mathbf{x}(t), \end{aligned} \quad (2.5)$$

where we now have a length- d vector $\mathbf{x}(t) = (x, \dot{x}, \ddot{x}, \dots)^T$ of the latent process $x(t)$ and its first $d - 1$ time derivatives. $\mathbf{F}$, called the feedback matrix, is a $d \times d$ matrix encoding the state evolution. The noise effect matrix, $\mathbf{L}$, is a column vector that determines which of the state's derivatives are driven by the white noise process $\mathbf{w}(t)$, which is itself defined by its spectral density $\mathbf{Q}_c$,

$$\mathbb{E}[\mathbf{w}(t)\mathbf{w}(t')^T] = \mathbf{Q}_c \delta_{\text{dirac}}(t - t'). \quad (2.6)$$

Finally, the observation matrix $\mathbf{H}_k$ projects the latent state vector at given time $\mathbf{x}(t_k)$ to the observed space $f(t_k)$. When dealing with instantaneous GP kernels, and if only the state x is measured, then $\mathbf{H}_k$ is simply a constant row vector $[1, 0, \dots, 0]$ that picks out the latent state x . Provided they exist, derivatives of $x(t)$ can be observed by simply changing the corresponding elements of $\mathbf{H}_k$ to a 1. Likewise, if we are interested in the sum of the latent state and its first derivative, the observation matrix becomes $\mathbf{H}_k = [1, 1, 0, \dots, 0]$. Examples of such a GP model are explored in Aigrain et al. 2012, Rajpaul et al. 2015, Jones et al. 2022, Gilbertson et al. 2020, and Tran et al. 2023. In general, $\mathbf{H}_k$ may vary from

data point to data point, or might include an amplitude term (which could be different depending on which instrument made the measurement). We can also apply linear operators to $\mathbf{H}$ and preserve the linear Gaussian behavior of the SSM (Särkkä et al. 2013).

Like before, our measurements¹² are noisy samples of the observed latent process

$$y_n = f(t_n) + \epsilon_n = \mathbf{H}_n \mathbf{x}(t_n) + \epsilon_n, \quad (2.7)$$

where the measurement noise ϵ_n is the same as before.

2.2.1. The GP-SSM equivalence

With the observation model in the state-space defined, the question now is how to go from a GP kernel function to an SDE (Eq. 2.5). Here, we provide a top-level overview; for more details and an example derivation for the SHO kernel, see Appendix B. Section 4 in Hartikainen & Särkkä (2010), which does the same for the Matérn family and squared exponential kernels, is also useful.

For a desired GP kernel, the matrices $\mathbf{F}$, $\mathbf{L}$, and $\mathbf{Q}_c$ are derived from the PSD, $S(\omega)$, which is the Fourier

¹² Here we assume 1D data, i.e. y_n is a scalar. Though, as long as $\mathbf{H}$ etc. (see Table 1) are appropriately shaped, multivariate data with dimension D is perfectly compatible here.

dual of the kernel function (per the Wiener–Khinchin theorem; [Rasmussen & Williams 2006](#))

$$S(\omega) = \int_{-\infty}^{\infty} k(\Delta) e^{-i\omega\Delta} d\Delta. \quad (2.8)$$

If the PSD is a rational function, then we can write it as the output of a linear time-invariant (LTI) system where the input is white noise with amplitude q (Theorem 3.4 in [Anderson & Moore 1979](#)),

$$S(\omega) = |H(i\omega)|^2 q, \quad (2.9)$$

where $H(i\omega)$, called the transfer function, determines how the system responds to the white noise input. The appropriate transfer function $H(s)$, in the complex s -domain, can be identified through spectral factorization (see Ch. 9 in [Anderson & Moore 1979](#)). By definition, the transfer function is the ratio of the output to the input of a system in the frequency domain,

$$H(s) \equiv \frac{X(s)}{W(s)}, \quad (2.10)$$

with $X(s)$ and $W(s)$ being the Laplace transforms of $x(t)$ and $w(t)$ (the output and the input, respectively). One can choose q such that $H(s)$ can be written as

$$H(s) = \frac{1}{s^d + a_{d-1}s^{d-1} + \dots + a_0}, \quad (2.11)$$

where a_i are coefficients. This form is convenient because it lets us rearrange Eq. 2.10 into

$$W(s) = s^d F(s) + a_{d-1}s^{d-1}F(s) + \dots + a_0F(s), \quad (2.12)$$

which can be converted to the time domain by inverse Laplace transforming both sides, turning multiplication by s in the Laplace domain into time derivatives in the time domain, thus giving the SDE

$$\frac{d^d x(t)}{dt^d} + a_{d-1} \frac{d^{d-1} x(t)}{dt^{d-1}} + \dots + a_0 x(t) = w(t). \quad (2.13)$$

Finally, the SDE can be put into matrix form by constructing the matrices $\mathbf{F}$ and $\mathbf{L}$:

$$\mathbf{F} = \begin{pmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ -a_0 & -a_1 & \dots & \dots & -a_{d-1} \end{pmatrix}, \quad \mathbf{L} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}, \quad (2.14)$$

yielding Eq. 2.5.

2.2.2. Solution to the SDE

To solve the SDE, we first need to define the initial conditions of the process. These are determined by the mean of the process (which we take to be zero without loss of generality) and the prior covariance, also called the stationary covariance $\mathbf{P}_\infty$. $\mathbf{P}_\infty$ is defined as the covariance after the system has settled, i.e. $t \rightarrow \infty$ so $d\mathbf{P}/dt = 0$. It has the following definition in the state-space formalism as the solution to the continuous-time Lyapunov equation (see Chapters 5.5, 6.1, & 6.5 in [Särkkä & Solin 2019](#), for the derivation from the expected covariance of the SDE in the stationary limit),

$$\frac{d\mathbf{P}}{dt} = \mathbf{F}\mathbf{P}_\infty + \mathbf{P}_\infty\mathbf{F}^T + \mathbf{L}\mathbf{Q}_c\mathbf{L}^T = 0. \quad (2.15)$$

Given $\mathbf{F}$ and $\mathbf{L}$, one can solve Eq. 2.15 for the elements of $\mathbf{P}_\infty$ in terms of $\mathbf{Q}_c$. The result should be a diagonal matrix. The first element corresponds to the stationary covariance of the latent process, i.e. the kernel function evaluated at zero time-lag, $k(0)$. This equivalence allows one to define $\mathbf{Q}_c$ in terms of the kernel parameters, and subsequently $\mathbf{P}_\infty$ is determined. One will also find that the lone element of $\mathbf{Q}_c$ is the same q from Eq. 2.9. See Appendix B for a worked example with the SHO kernel.

The solution to Eq. 2.5 at time t relative to another time t_0 is

$$\mathbf{x}(t) = \Phi(t - t_0)\mathbf{x}(t_0) + \int_{t_0}^t \Phi(t - \tau)\mathbf{L}w(\tau) d\tau, \quad (2.16)$$

where the transition matrix

$$\Phi(t - t_0) = \exp(\mathbf{F}(t - t_0)) \quad (2.17)$$

is the matrix exponential of the feedback matrix times the relative time difference. To determine the conditional solutions given the data, we can discretize Eq. 2.16 at time t_{k+1} relative to t_k as

$$\mathbf{x}_{k+1} = \mathbf{A}_k\mathbf{x}_k + \mathbf{q}_k, \quad \mathbf{q}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k) \quad (2.18)$$

where $\mathbf{x}_k \equiv \mathbf{x}(t_k)$, $\mathbf{A}_k = \Phi(\Delta_k)$, and $\Delta_k = t_{k+1} - t_k$. The matrix $\mathbf{Q}_k = E[\mathbf{q}_k\mathbf{q}_k^T]$ is called the process noise covariance and is given by substituting Eq. 2.6 into the second term of Eq. 2.16

$$\begin{aligned} \mathbf{Q}_k &= \int_0^{\Delta_k} \Phi(\Delta_k - \tau)\mathbf{L}\mathbf{Q}_c\mathbf{L}^T\Phi(\Delta_k - \tau)^T d\tau \\ &= \mathbf{P}_\infty - \mathbf{A}_k\mathbf{P}_\infty\mathbf{A}_k^T. \end{aligned} \quad (2.19)$$

The last equality, which can be found in [Solin & Särkkä \(2014b\)](#) and [Särkkä & Solin \(2019\)](#), is preferred over working out the Lyapunov integral for its speed and numerical stability, especially when $\mathbf{P}_\infty$ and $\mathbf{A}_k$ are

known analytically. Alternatively, the integral can be efficiently computed by taking the matrix exponential of a block upper-triangular matrix involving $\mathbf{F}$, $\mathbf{L}\mathbf{Q}_c\mathbf{L}^T$, and Δ , from which the product of two submatrices gives exactly $\mathbf{Q}_k$ (Van Loan 1978, see Eq. B13 for an example). However, for Δ around $\sim 10^3$ times longer than the kernel timescale, the numerical value of this exponential is too large to be represented numerically; for data with large gaps, the analytic forms are required.

For a pair of points t and t' , the GP covariance function is (Eq. 4 in Hartikainen & Särkkä 2010)

$$k(t, t') = \begin{cases} \mathbf{H}(t')\mathbf{P}_\infty\Phi(|t - t'|)^T\mathbf{H}(t)^T, & t \geq t', \\ \mathbf{H}(t)\Phi(|t - t'|)\mathbf{P}_\infty\mathbf{H}(t')^T, & t < t'. \end{cases} \quad (2.20)$$

To generate predictions or compute likelihoods, because everything is Gaussian, we need only track the means and variances of these $\mathbf{x}_k$ from state to state. A vast literature anchored in control theory exists for doing just that as countless real-world problems can be modeled as state-space systems. The Kalman filter and Rauch–Tung–Striebel (RTS) smoothing algorithms, discussed further in the following sections, yield the optimal predictions for such linear Gaussian systems; these optimal predictions are identical to the GP conditioned mean and variance. See Särkkä & Svensson (2023) (Särkkä & Solin 2019, for the continuous case) for the full derivations or Figure 1 for a numerical demonstration.

2.3. GP conditioning

The problem of conditioning the GP is to derive the optimal prediction for the mean, $\mathbf{m}$, and covariance, $\mathbf{P}$, of the state $\mathbf{x}$ given the data $\mathbf{y}$. For a linear Gaussian SSM, the solution is given by Bayesian filtering and smoothing algorithms (Särkkä & Svensson 2023). Formally, for a 1D time series, these algorithms scale as $\mathcal{O}(Nd^3)$ because they involve matrix operations (multiplications, inversions) on (at largest) $d \times d$ matrices per iteration through the N data points. For the SSMs we consider, $d = 2$ or 3 , so for realistic datasets $N \gg d$ and so $\mathcal{O}(Nd^3) \simeq \mathcal{O}(N)$.

2.3.1. Kalman filter

The Kalman filter (Kalman 1960) computes the conditional probabilities at each state given all previous data, $p(\mathbf{x}_k|\mathbf{y}_{1:k})$. Since everything is linear and Gaussian, we just need to compute the mean and covariance at each state, iterating chronologically through the data. To start the iteration, we initialize the state to $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{m}_0, \mathbf{P}_0)$ with $\mathbf{m}_0$ set to the kernel mean function (usually zero for the state and all its derivatives)

and $\mathbf{P}_0$ set to the stationary covariance ($\mathbf{P}_\infty$). The algorithm is then (Theorem 6.6 in Särkkä & Svensson 2023)

Prediction:

$$\begin{aligned} \mathbf{m}_k^- &= \mathbf{A}_{k-1}\mathbf{m}_{k-1}, \\ \mathbf{P}_k^- &= \mathbf{A}_{k-1}\mathbf{P}_{k-1}\mathbf{A}_{k-1}^T + \mathbf{Q}_{k-1}. \end{aligned} \quad (2.21)$$

Update:

$$\begin{aligned} v_k &= y_k - \mathbf{H}_k\mathbf{m}_k^-, \\ \mathbf{S}_k &= \mathbf{H}_k\mathbf{P}_k^-\mathbf{H}_k^T + \mathbf{R}_k, \\ \mathbf{K}_k &= \mathbf{P}_k^-\mathbf{H}_k^T\mathbf{S}_k^{-1}, \\ \mathbf{m}_k &= \mathbf{m}_k^- + \mathbf{K}_kv_k, \\ \mathbf{P}_k &= \mathbf{P}_k^- - \mathbf{K}_k\mathbf{S}_k\mathbf{K}_k^T. \end{aligned} \quad (2.22)$$

The prediction step (Eq. 2.21) can be thought of as simply transitioning from the previous (filtered) state to the current state. The update step (Eq. 2.22) first calculates the “surprise term” v_k (also called the innovation), which is simply the difference between our prediction in the observed space and the actual measured value. The uncertainty in the prediction, $\mathbf{S}_k$ (also called the innovation covariance), defines the Kalman gain ($\mathbf{K}_k$) which effectively weights the surprise term for the purpose of updating our predicted mean $\mathbf{m}_k$ and variance $\mathbf{P}_k$. The filtered $\mathbf{m}_k$ and $\mathbf{P}_k$ are then carried to the next iteration, continuing until $k = N$. Note that these matrices are all in the state-space. That is, the $\mathbf{m}$ s have shape $d \times 1$ and the $\mathbf{P}$ s have shape $d \times d$. Only the innovation step projects the mean state to the observed space ($\mathbf{H}_k\mathbf{m}_k$) to compare to the measured y_k .

2.3.2. RTS smoother

The RTS (Rauch et al. 1965) smoothing algorithm refines these predictions using future data to give $p(\mathbf{x}_k|\mathbf{y}_{1:N})$ for all k by applying the transition matrix to the Kalman filter results in reverse-chronological order. It is important to note that this is not the same as Kalman filtering in reverse as causality assumptions in the noise model must be obeyed (e.g. driving and damping backwards in time is not the same as undriving and undamping).

The final Kalman-filtered state is already informed by all other data points; as such, it is already smoothed. That is, $\hat{\mathbf{m}}_N = \mathbf{m}_N$ and $\hat{\mathbf{P}}_N = \mathbf{P}_N$. Then, starting from the penultimate ($k = N - 1$) state and iterating in reverse-chronological order, the RTS algorithm is (The-

orem 12.2 in Särkkä & Svensson 2023)

$$\begin{aligned}
 \mathbf{m}_{k+1}^- &= \mathbf{A}_k \mathbf{m}_k, \\
 \mathbf{P}_{k+1}^- &= \mathbf{A}_k \mathbf{P}_k \mathbf{A}_k^T + \mathbf{Q}_k, \\
 \mathbf{G}_k &= \mathbf{P}_k \mathbf{A}_k^T [\mathbf{P}_{k+1}^-]^{-1}, \\
 \hat{\mathbf{m}}_k &= \mathbf{m}_k + \mathbf{G}_k [\hat{\mathbf{m}}_{k+1} - \mathbf{m}_{k+1}^-], \\
 \hat{\mathbf{P}}_k &= \mathbf{P}_k + \mathbf{G}_k [\hat{\mathbf{P}}_{k+1} - \mathbf{P}_{k+1}^-] \mathbf{G}_k^T. \quad (2.23)
 \end{aligned}$$

Note $\mathbf{m}_k$, $\mathbf{P}_k$, $\mathbf{m}_{k+1}^-$, and $\mathbf{P}_{k+1}^-$ are already computed by the Kalman filter (Eqs. 2.21, 2.22). $\mathbf{G}_k$ is the “smoothing gain” which weights “how much we got the $k+1^{th}$ state’s prediction wrong” by the data-conditioned variance in the process between $k+1 \rightarrow k$, to “correct” the Kalman-filtered state k . The smoothed results are mathematically equivalent to full GP conditioning (Eq. 2.4) when the state-space SDE is linear and the driving noise is Gaussian (see Ch. 12.4 of Särkkä & Solin 2019, and references therein). That is, $\mathbf{H}\hat{\mathbf{m}} \equiv \boldsymbol{\mu}_{GP}$ and $\mathbf{H}\hat{\mathbf{P}}\mathbf{H}^T \equiv \boldsymbol{\Sigma}_{GP}$.

2.4. The log-likelihood

Byproducts of the Kalman filter are the ingredients to compute the likelihood (Eq. 16.5 in Särkkä & Svensson 2023),

$$p(\mathbf{y}_{1:N}|\boldsymbol{\theta}) = \prod_{n=1}^N p(y_n|\mathbf{y}_{1:n-1}, \boldsymbol{\theta}). \quad (2.24)$$

The individual $p(y_n|\mathbf{y}_{1:n-1}, \boldsymbol{\theta})$ are given by the first two pieces of the “update” step in Eq. 2.22, since

$$p(y_n|\mathbf{y}_{1:n-1}, \boldsymbol{\theta}) = N(\mathbf{H}_n \mathbf{m}_n^-, \mathbf{S}_n). \quad (2.25)$$

The total log-likelihood $\mathcal{L}(\mathbf{y}|\boldsymbol{\theta}) = \log p(\mathbf{y}_{1:N}|\boldsymbol{\theta})$ is thus

$$\mathcal{L}(\mathbf{y}|\boldsymbol{\theta}) = -\frac{1}{2} \sum_{n=1}^N (\log \det(2\pi \mathbf{S}_n) + \mathbf{v}_n^T \mathbf{S}_n^{-1} \mathbf{v}_n). \quad (2.26)$$

Eq. 2.26 can be input to the usual methods (e.g. gradient descent, Markov chain Monte Carlo) for hyperparameter optimization.

2.5. Predicting at arbitrary times

It is straightforward to extend the Kalman/RTS algorithms to predict the mean and covariance at an arbitrary time t_* . This is related to the idea of “fast sampling” described in Section 4.6 of Särkkä & Svensson (2023). There are three cases:

1. *Retrodiction.* ($t_* < t_1$): If the test point is before the data, compute the RTS smoothed estimate at t_* from the first measurement’s smoothed

($\hat{\mathbf{m}}_1, \hat{\mathbf{P}}_1$) and predicted ($\mathbf{m}_1^-, \mathbf{P}_1^-$) states. The “filtered” mean at the test point, $\mathbf{m}_*$, is set to the kernel mean (e.g. zero) and the covariance $\mathbf{P}_*$ is the stationary covariance $\mathbf{P}_\infty$.

2. *Interpolation.* ($t_1 < t_* < t_N$): If the test point is during the data, we use the usual Kalman prediction step from the most recent data point’s Kalman-filtered state and then refine the prediction with a RTS smoothing step from the nearest future data point’s smoothed state.
3. *Extrapolation.* ($t_* > t_N$): If the test point is after the data, we simply use the Kalman prediction from the final data point’s filtered/smoothed state.

Algorithm 1: SSM prediction at arbitrary time t_*

```

1:  $t_1 \leftarrow \min(t)$ 
2:  $t_N \leftarrow \max(t)$ 
3: if  $t_* < t_1$  then ▷ Retrodict
4:    $\Delta_* \leftarrow t_1 - t_*$ 
5:    $\mathbf{A}_* \leftarrow \Phi(\Delta_*)$ 
6:    $\mathbf{m}_0 \leftarrow \mathbf{0}$ 
7:    $\mathbf{P}_0 \leftarrow \mathbf{P}_\infty$ 
8:    $\mathbf{G}_0 \leftarrow \mathbf{P}_0 \mathbf{A}_*^T [\mathbf{P}_1^-]^{-1}$ 
9:    $\hat{\mathbf{m}}_* \leftarrow \mathbf{m}_0 + \mathbf{G}_0 [\hat{\mathbf{m}}_1 - \mathbf{m}_1^-]$ 
10:   $\hat{\mathbf{P}}_* \leftarrow \mathbf{P}_0 + \mathbf{G}_0 [\hat{\mathbf{P}}_1 - \mathbf{P}_1^-]$ 
11: else if  $t_1 < t_* < t_N$  then ▷ Interpolate
12:   $t_{\text{prev}} \leftarrow \max(t|t < t_*)$ 
13:   $t_{\text{next}} \leftarrow \min(t|t > t_*)$ 
14:   $\mathbf{A}_{\text{prev}} \leftarrow \Phi(t_* - t_{\text{prev}})$ 
15:   $\mathbf{A}_{\text{next}} \leftarrow \Phi(t_{\text{next}} - t_*)$ 
16:   $\mathbf{Q}_{\text{prev}} \leftarrow \mathbf{Q}(t_* - t_{\text{prev}})$ 
17:   $\mathbf{m}_* \leftarrow \mathbf{A}_{\text{prev}} \mathbf{m}_{k-1}$ 
18:   $\mathbf{P}_* \leftarrow \mathbf{A}_{\text{prev}} \mathbf{P}_{k-1} \mathbf{A}_{\text{prev}}^T + \mathbf{Q}_{\text{prev}}$ 
19:   $\mathbf{G}_* \leftarrow \mathbf{P}_0 \mathbf{A}_{\text{next}}^T [\mathbf{P}_1^-]^{-1}$ 
20:   $\hat{\mathbf{m}}_* \leftarrow \mathbf{m}_* + \mathbf{G}_* [\hat{\mathbf{m}}_{\text{next}} - \mathbf{m}_{\text{next}}^-]$ 
21:   $\hat{\mathbf{P}}_* \leftarrow \mathbf{P}_* + \mathbf{G}_* [\hat{\mathbf{P}}_{\text{next}} - \mathbf{P}_{\text{next}}^-]$ 
22: else if  $t_* > t_N$  then ▷ Extrapolate
23:   $\Delta_* \leftarrow t_* - t_N$ 
24:   $\mathbf{A}_* \leftarrow \Phi(\Delta_*)$ 
25:   $\mathbf{Q}_* \leftarrow \mathbf{Q}(\Delta_*)$ 
26:   $\hat{\mathbf{m}}_* \leftarrow \mathbf{A}_* \mathbf{m}_N$ 
27:   $\hat{\mathbf{P}}_* \leftarrow \mathbf{A}_* \mathbf{P}_N \mathbf{A}_*^T + \mathbf{Q}_*$ 
28: end if

```

Algorithm 1 demonstrates this in pseudocode. An alternative algorithm was described in the appendix of Kelly et al. (2014) using a linearized form for the predicted state at the test point to derive their smoothing equations. Though it is significantly more computationally expensive because it recomputes the smoothing gain at every test point, which in each case involves a loop through all the future data to that test point.

In the GP framework, predicting at M arbitrary test points involves rectangular matrix multiplications; these

can still be $\mathcal{O}(N)$ for QSMs (e.g. Algorithm 5 in [Pernet & Storjohann 2018](#), see also [Pernet et al. 2023](#)). Generally, Eq. 2.4 for GPs (QSM or dense) scales linearly with the number of test points M , though the implicit conditioning still grows with N^2 (or N for QSMs). By contrast, Algorithm 1 is embarrassingly parallel; each prediction only depends on the nearest (past and/or future) data point. It is also significantly less memory intensive (see e.g., Figure 4).

To demonstrate the SSM–GP equivalence, we numerically validated the SSM approach (using `smolgp`; [Rubenzahl & Hattori 2026](#)) to the GP approach (using `tinygp`; [Foreman-Mackey et al. 2024](#)) for a multicomponent (see Appendix C) kernel made from the sum of an SHO and a Matérn-5/2. We took a random realization of this process as our true signal, from which we generated synthetic measurements. We then conditioned an SSM and GP on these data and made predictions at a high-resolution test grid. The results of conditioning and predicting (for the full model and for each component kernel), as well as the log-likelihood, were equivalent to within machine precision. Figure 1 shows this comparison.

3. INTEGRATED OBSERVATIONS

3.1. In the GP framework

Handling exposure-integrated observations, especially when observations are allowed to overlap, requires significant bookkeeping. In the traditional GP framework, the cross-covariance between two observations at times t_1 and t_2 with corresponding exposure times δ_1 and δ_2 is

$$k_{FF}(t_1, t_2, \delta_1, \delta_2) = \frac{1}{\delta_1 \delta_2} \int_{t_1 - \delta_1/2}^{t_1 + \delta_1/2} \int_{t_2 - \delta_2/2}^{t_2 + \delta_2/2} k(t, t') dt dt'. \quad (3.1)$$

For the cases of fully separated or perfectly overlapping observations (such as an observation with itself, i.e. the diagonal), [Luhn et al. \(2026\)](#) (hereafter L26) derived a closed-form analytic solution for the SHO kernel. These two cases can then be combined with appropriate weighting along the nonoverlapping and shared segments of partially overlapping observations to correctly recover the joint covariance for arbitrary overlap. However, the logic to implement this in practice prevents the use of quasi-separable linear algebra, even if the underlying kernel is stationary and quasi-separable, for the simple reason that while the overlapping observations live near to the diagonal (in a time-ordered matrix), they lack an exploitable structure; they also do not form a LEAF ([Delisle et al. 2020](#)) matrix.

A datastream from a single instrument, usually¹³, does not have any overlapping observations. If so, and if the exposure time is constant for all data points, it can in some cases be possible to represent the integrated GP kernel in quasi-separable form. For example, an integrated SHO kernel can be expressed as the sum of four SHO kernels at shifted times and modified amplitudes, plus two constant terms, each of which is compatible with a quasi-separable form. However, combining data from multiple instruments during shared observing windows will often yield overlapping pairs of observations. In that case, the GP formalism is cursed to rely on dense matrix computations. In general though, for realistic datasets, overlapping observations represent a small fraction of the total number of data points (and certainly a small fraction of all the possible pairs of observations), so the dense matrix representation (while formally correct) is an inefficient way to deal with this problem.

3.2. In the SSM framework

SSMs are commonly used to model binned measurements in the context a fast-rate process that is occasionally sampled at a “slow-rate,” where each observed sample is the average of some number of (unobserved) fast-rate samples (e.g., [Guo & Huang 2015](#); [Fatehi & Huang 2017](#); [Salehi et al. 2018](#)), or as the integral of a continuous process during the interval between measurements ([Qian & Chang 2021](#)). In our case, we wish to draw an exact equivalence to L26, i.e. we have infrequent integrated samples of a continuous function where our measurements may overlap with one another. Our approach, based on footnote 1 of [Yaghoobi & Särkkä \(2025\)](#), essentially converts the method of L26 into an $\mathcal{O}(N)$ algorithm by integrating the dynamics rather than the covariance.

Let y_n be the exposure-averaged measurement at observation $n \in [1, N]$,

$$y_n = \frac{1}{\delta_n} \int_{t_n^s}^{t_n^e} \mathbf{H}_n \mathbf{x}(t) dt + \epsilon_n. \quad (3.2)$$

where t_n^s and t_n^e are the start and end times of the exposure, which has length $\delta_n = t_n^e - t_n^s$. Because integration is a linear operator, the measurement model remains linear and so is still described by a SSM ([Särkkä et al.](#)

¹³ An infrared echelle spectrograph might nondestructively (“up-the-ramp”) read-out during an exposure up to a fixed S/N independently across a number of pixel channels, yielding a different exposure time for each spectral segment. Additionally, adjacent echelle orders often cover overlapping wavelength ranges, so in such a scenario there can be simultaneous, overlapping measurements of the same signal from the same detector.

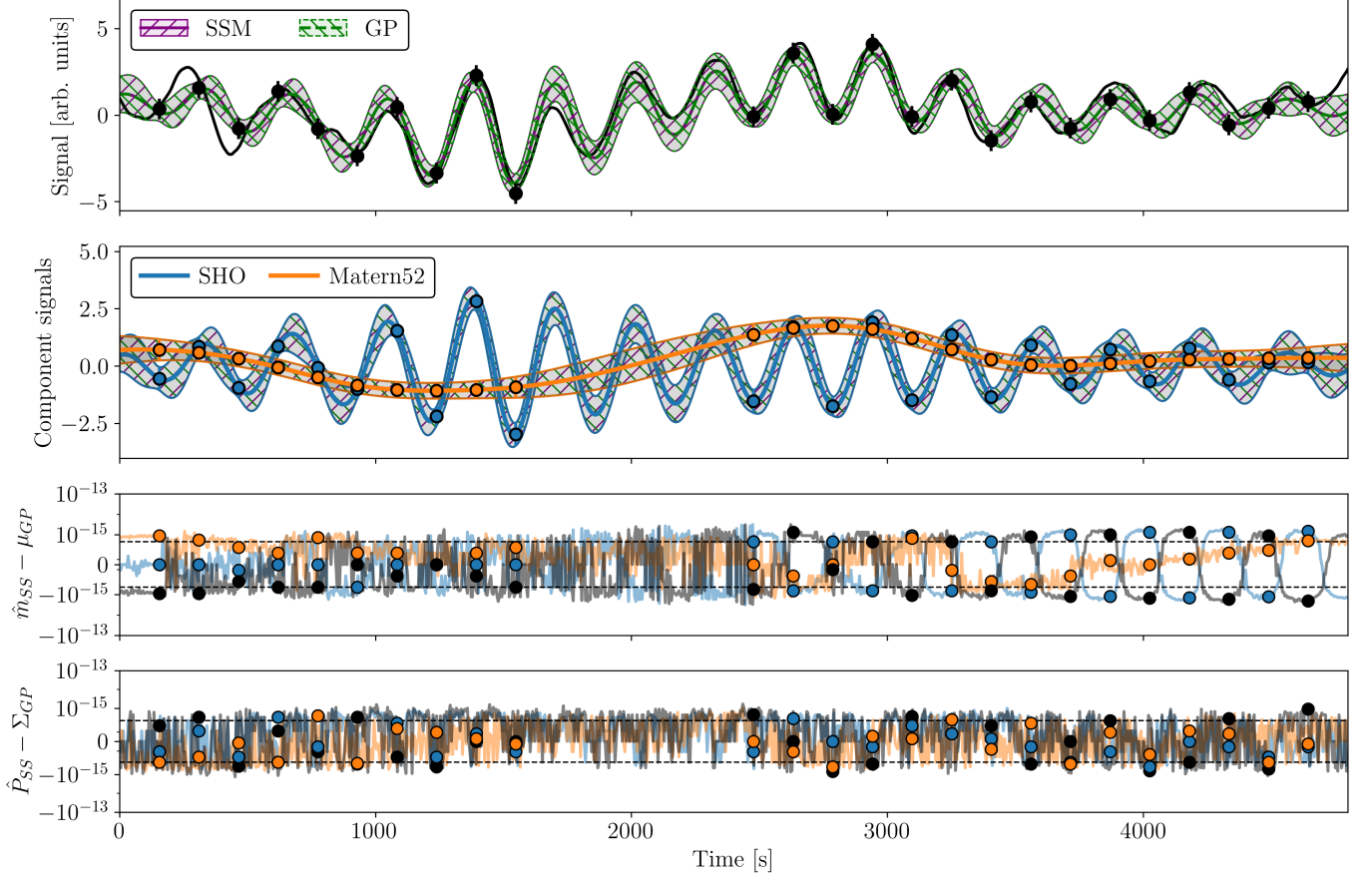


Figure 1. Equivalence of linear Gaussian SSMs to GPs. **Top:** The true signal (in black) is the sum of an SHO and Matérn-5/2 kernel. Synthetic measurements (black points) are noisy samples of this process. The purple “/” band shows the predicted mean and 1σ variance given the data from the SSM method; the green “\” band shows the same for the full GP method. **Middle:** The same as the top panel but decomposed into each component kernel’s prediction at the test points (filled bands) and at the data (points). **Bottom:** The bottom two panels show the residuals for the predicted mean and variance from the overall model prediction (black line) and component predictions (blue/orange lines) at the test points, and well as at the data (points), between the SSM method and the full GP method. The horizontal dashed lines denote machine precision for 64-bit floating-point numbers in the dynamic range of the data; In all cases, the difference between `smolgp` and `tinygp` is within this level.

2013). Moreover, our state is already the joint state of the instantaneous latent state and its first $d - 1$ time derivatives. We can thus simply introduce the integral state z such that

$$\frac{dz}{dt} = x. \quad (3.3)$$

We can then augment the SSM to be of the joint state $\tilde{\mathbf{x}} = [\mathbf{x}; z]$, where $\mathbf{x}$ is as before. The augmented SDE becomes

$$\frac{d\tilde{\mathbf{x}}}{dt} = \tilde{\mathbf{F}}\tilde{\mathbf{x}}(t) + \tilde{\mathbf{L}}\mathbf{w}(t), \quad (3.4)$$

with the augmented matrices

$$\tilde{\mathbf{F}} = \begin{pmatrix} \mathbf{F} & 0 \\ 1 & 0 \dots \end{pmatrix}, \quad \tilde{\mathbf{L}} = \begin{pmatrix} \mathbf{L} \\ 0 \end{pmatrix}. \quad (3.5)$$

The augmented state dimension becomes $d + 1$. **One can verify this gives the correct system of SDEs,**

$$\frac{d\tilde{\mathbf{x}}}{dt} = \begin{pmatrix} \frac{d\mathbf{x}}{dt} \\ \frac{dz}{dt} \end{pmatrix} = \tilde{\mathbf{F}}\tilde{\mathbf{x}} + \tilde{\mathbf{L}}\mathbf{w} = \begin{pmatrix} \mathbf{F}\mathbf{x} \\ x \end{pmatrix} + \begin{pmatrix} \mathbf{L}\mathbf{w} \\ 0 \end{pmatrix}. \quad (3.6)$$

The corresponding augmented transition matrix is

$$\tilde{\Phi}(\Delta) = \begin{pmatrix} \Phi(\Delta) & 0 \\ \tilde{\Phi}_x(\Delta) & 1 \end{pmatrix}, \quad \tilde{\Phi}(\Delta) = \int_0^\Delta \Phi(t) dt, \quad (3.7)$$

where $\tilde{\Phi}_x(\Delta)$ is a $1 \times d$ vector containing the integral, over the interval Δ , of the top row of the transition matrix; i.e., the integral of the transition in x and each of its derivatives. In other words, $\mathbf{x}$ evolves according to the instantaneous dynamics like normal, while z is accumulated over the transition. We can verify that Eq. 3.7 gives the correct behavior by checking the solution to

the SDE matches Eq. 2.16,

$$\begin{aligned}\tilde{\mathbf{x}}(t) &= \tilde{\Phi}(t - t_0)\tilde{\mathbf{x}}(t_0) + \int_{t_0}^t \tilde{\Phi}(t - \tau)\tilde{\mathbf{L}}w(\tau) d\tau \\ \begin{pmatrix} \mathbf{x}(t) \\ z(t) \end{pmatrix} &= \begin{pmatrix} \Phi(t - t_0)\mathbf{x}(t_0) + \int_{t_0}^t \Phi(t - \tau)\mathbf{L}w(\tau) d\tau \\ z(t_0) + \tilde{\Phi}(t - t_0)\mathbf{x}(t_0) + \int_{t_0}^t \tilde{\Phi}(t - \tau)\mathbf{L}w(\tau) d\tau \end{pmatrix}.\end{aligned}\quad (3.8)$$

In practice, one can simply compute $\tilde{\Phi}(\Delta)$ numerically as the matrix exponential of $\tilde{\mathbf{F}}\Delta$. However, it is more numerically stable to use the method of Van Loan (1978) to get $\tilde{\Phi}(\Delta) = \mathbf{G}_3$, where

$$\mathbf{C} = \begin{pmatrix} \mathbf{F} & \mathbf{I} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \rightarrow \exp(\mathbf{C}\Delta) = \begin{pmatrix} \mathbf{F}_3 & \mathbf{G}_3 \\ \mathbf{0} & \mathbf{F}_4 \end{pmatrix}, \quad (3.9)$$

where $\mathbf{F}_3$ and $\mathbf{F}_4$ are other block matrices irrelevant to the definition of $\tilde{\Phi}$. That is, one can extract only $\mathbf{G}_3 \equiv \tilde{\Phi}(\Delta)$ and then assemble $\tilde{\Phi}$ via Eq. 3.7. Of course, for certain kernels, the $\tilde{\Phi}$ integral may be doable analytically (e.g., Eq. B12); this is preferred whenever possible for numerical stability over long transitions.

The augmented process noise is

$$\begin{aligned}\tilde{\mathbf{Q}} &= \int_0^\Delta \tilde{\Phi}(t)\tilde{\mathbf{L}}\mathbf{Q}_c\tilde{\mathbf{L}}^T\tilde{\Phi}(t)^T dt \\ &= \begin{pmatrix} \int_0^\Delta \Phi\mathbf{L}\mathbf{Q}_c\mathbf{L}^T\Phi^T dt & \int_0^\Delta \Phi\mathbf{L}\mathbf{Q}_c\mathbf{L}^T\tilde{\Phi}^T dt \\ \int_0^\Delta \tilde{\Phi}\mathbf{L}\mathbf{Q}_c\mathbf{L}^T\Phi^T dt & \int_0^\Delta \tilde{\Phi}\mathbf{L}\mathbf{Q}_c\mathbf{L}^T\tilde{\Phi}^T dt \end{pmatrix},\end{aligned}\quad (3.10)$$

where Φ and $\tilde{\Phi}$ are both functions of t , though we omitted writing that explicitly for notational compactness. Each of these components has its own representation as a Van Loan matrix exponential (Van Loan 1978), or $\tilde{\mathbf{Q}}$ can be computed at once from $\tilde{\mathbf{F}}$ and $\tilde{\mathbf{L}}\mathbf{Q}_c\tilde{\mathbf{L}}^T$ (see Eq. B13). In practice, because the matrix exponential is numerically unstable for large Δ (i.e., long gaps in the data set), we do the former, so that the top-left block ($\mathbf{Q}$ for the base SSM) can be populated by its analytic solution. While the other blocks are unstable for large Δ , in practice they are immediately reset at the end of the gap so the result is unaffected. If there are indeed long ($\Delta \gtrsim 10^5$ time units) integration intervals in the measurements, then Eq. 3.10 must be defined analytically by working out the integrals; the identity in Eq. 2.19 involving $\mathbf{P}_\infty$ and $\mathbf{A}$ cannot be used to obtain $\tilde{\mathbf{Q}}$ because the z state is a zero-variance process (i.e., $\tilde{\mathbf{F}}$ is noninvertible).

With the augmented model fully defined, we can discretize at the starts and ends of every exposure, yielding $K = 2N$ total states in total. Only the states $\tilde{\mathbf{x}}_k$ such that t_k corresponds to the end of an exposure (with matching timestamp t_n^e) will have measurements (y_n) for the usual Kalman update step. Conversely, the states $\tilde{\mathbf{x}}_k$

such that t_k is the start of an exposure (with timestamp t_n^s) will have its z state reset to zero; this is effectively the “update” step in that we “measure” nothing at the start of an exposure. That is, the measurement model is

$$\begin{aligned}0 &= z(t_k) && \text{if } t_k = t_n^s \text{ is an exposure start,} \\ y_n &= \tilde{\mathbf{H}}_n\tilde{\mathbf{x}}_k + \epsilon_n && \text{if } t_k = t_n^e \text{ is an exposure end,}\end{aligned}\quad (3.11)$$

where

$$\tilde{\mathbf{H}}_n = \frac{1}{\delta_n} \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix} \quad (3.12)$$

picks out the integral state in $\tilde{\mathbf{x}}_k = [x, \dot{x}, z]^T$, which represents the integral from t_n^s to t_n^e (because we reset the integral state to zero at t_n^s) and divides by the exposure length δ_n to return the average of the state over that interval, which we measure with noise ϵ_n . In other words, $y_n = z(t_n^e)/\delta_n + \epsilon_n$.

With Eqs. 3.4 and 3.11 being of the form of Eqs. 2.5 and 2.7, the augmented model with the integral state defines a state-space model which we can use the machinery of Section 2.2 to solve. Next, we derive the Kalman and RTS equations to compute the predicted, filtered, and smoothed means and covariances at the $K = 2N$ states. These can then be used in Algorithm 1 as before to make predictions at arbitrary times.

3.2.1. Integrated Kalman Filter

Define the reset matrix $\mathcal{R}$ to be

$$\mathcal{R} = \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \quad (3.13)$$

so that the $\mathbf{x}$ state is preserved but the z state is zeroed out. The only change to the Kalman filter is to apply the reset matrix as a transition matrix in the “update” step for exposure-start states. This can equivalently be thought of as a two-step transition to before and after the reset, where at each step we lack a measurement and so the update is skipped. We thus have (for exposure-start states),

Prediction:

$$\begin{aligned}\mathbf{m}_k^- &= \tilde{\mathbf{A}}_{k-1}\mathbf{m}_{k-1}, \\ \mathbf{P}_k^- &= \tilde{\mathbf{A}}_{k-1}\mathbf{P}_{k-1}\tilde{\mathbf{A}}_{k-1}^T + \tilde{\mathbf{Q}}_{k-1}.\end{aligned}\quad (3.14)$$

Update:

$$\begin{aligned}\mathbf{m}_k &= \mathcal{R}\mathbf{m}_k^-, \\ \mathbf{P}_k &= \mathcal{R}\mathbf{P}_k^-\mathcal{R}^T.\end{aligned}\quad (3.15)$$

In other words, the predicted exposure-start state is the same as the usual Kalman prediction from the previous

state. We then treat the filtered exposure-start state as a deterministic ($\mathbf{Q}_k = 0$) transition from that prediction, taking the reset matrix as our transition matrix. Exposure-end states are predicted and filtered according to the usual Kalman filter prescription (Eq. 2.21 and 2.22) with the augmented matrices.

This way, all the work to handle exposure integration is built-in to $\mathbf{z}$ and $\tilde{\mathbf{H}}_n$; $\mathbf{z}$ accumulates only during exposure intervals, at the end of which $\tilde{\mathbf{H}}_n$ projects the exposure-averaged value to the observed space.

3.2.2. Integrated RTS Smoother

RTS smoothing for exposure-end states have the same form as Eq. 2.23. For RTS smoothing at exposure-start states, we will have to smooth over both the usual Δ_k and also over the reset. As such, there are two changes to the usual RTS equations. The first is that, because $\hat{\mathbf{m}}_k, \hat{\mathbf{P}}_k$ are defined after smoothing over the reset, the corresponding quantities to smooth are $\mathbf{m}_k^-, \mathbf{P}_k^-$. The second is that, by interpreting the reset matrix as a transition matrix, we can think of forward transitions from $(\mathbf{m}_k^-, \mathbf{P}_k^-)$ to their successive state $(\mathbf{m}_{k+1}^-, \mathbf{P}_{k+1}^-)$ as a single transition where $\tilde{\mathbf{A}}_k \mathbf{R}$ is the transition matrix. Substituting these two changes into Eq. 2.23, we get the equations for smoothing an exposure-start state

$$\begin{aligned} \mathbf{G}_k &= \mathbf{P}_k^- (\tilde{\mathbf{A}}_k \mathbf{R})^T [\mathbf{P}_{k+1}^-]^{-1}, \\ \hat{\mathbf{m}}_k &= \mathbf{m}_k^- + \mathbf{G}_k [\hat{\mathbf{m}}_{k+1} - \mathbf{m}_{k+1}^-], \\ \hat{\mathbf{P}}_k &= \mathbf{P}_k^- + \mathbf{G}_k [\hat{\mathbf{P}}_{k+1} - \mathbf{P}_{k+1}^-] \mathbf{G}_k^T. \end{aligned} \quad (3.16)$$

Altogether, Eqs. 3.14, 3.15, and 3.16 yield, respectively, the predicted $(\mathbf{m}_k^-, \mathbf{P}_k^-)$, filtered $(\mathbf{m}_k, \mathbf{P}_k)$, and smoothed $(\hat{\mathbf{m}}_k, \hat{\mathbf{P}}_k)$ means and covariances at each of the $2N$ discretized states at all exposure starts and ends. The smoothed means and covariances at the exposure-end states, when projected through their corresponding $\tilde{\mathbf{H}}_n$, then give the conditioned mean $(\tilde{\mathbf{H}}_n \hat{\mathbf{m}}_n)$ and variance $(\tilde{\mathbf{H}}_n \hat{\mathbf{P}}_n \tilde{\mathbf{H}}_n^T)$ at each of the measurements. For making predictions at arbitrary times with Algorithm 1, all $2N$ states must be used for accurate interpolation between (or during) measurements.

3.2.3. Handling overlapping observations

We assume, without loss of generality¹⁴, that observations from a single instrument do not overlap. That is, for each instrument $i \in [1, N_{\text{inst}}]$, $t_n^e < t_{n+1}^s \forall n \in N_i$.

¹⁴ Any set of overlapping measurements can be “labeled” as belonging to different instruments such that each unique instrument label does not contain any self-overlaps. The Kalman/RTS algorithms then scale as $\mathcal{O}(N(d + N_{\text{inst}})^3)$

Then, we can introduce a z_i state for each instrument into the augmented model so that our state becomes $\tilde{\mathbf{x}} = [\mathbf{x}; z_1; z_2, \dots]^T$. The state dimension is now $d + N_{\text{inst}}$, and a new row and/or column is appended to each of the augmented matrices for each instrument,

$$\begin{aligned} \tilde{\mathbf{F}} &= \begin{pmatrix} \mathbf{F} & 0 \dots \\ & \vdots \\ 1 & 0 \dots \\ & \vdots \end{pmatrix}, \quad \tilde{\mathbf{L}} = \begin{pmatrix} \mathbf{L} \\ 0 \\ \vdots \end{pmatrix}, \\ \tilde{\Phi} &= \begin{pmatrix} \Phi & 0 & \dots \\ \Phi_x & 1 & \\ \vdots & & \ddots \end{pmatrix}, \quad \tilde{\mathbf{Q}} = \begin{pmatrix} \mathbf{Q} & \tilde{\mathbf{Q}}_{12} \dots \\ \tilde{\mathbf{Q}}_{21} & \tilde{\mathbf{Q}}_{22} \dots \\ \vdots & \vdots & \ddots \end{pmatrix}, \end{aligned} \quad (3.17)$$

with zeros elsewhere. In other words, $\tilde{\mathbf{F}}$ has $\mathbf{F}$ as an upper-left block matrix, ones along the first column below that, and zeros elsewhere. $\tilde{\mathbf{L}}$ has $\mathbf{L}$ stacked above a zero for each instrument. $\tilde{\Phi}$ has Φ as an upper-left block matrix, Φ_x at the start of each row below that, ones along the diagonal, and zeros elsewhere. $\tilde{\mathbf{Q}}$ has $\mathbf{Q}$ as a block matrix in the upper-left, $\tilde{\mathbf{Q}}_{21}$ at the start of every row below that, $\tilde{\mathbf{Q}}_{12}$ repeatedly appended to the first column after $\mathbf{Q}$, and $\tilde{\mathbf{Q}}_{22}$ everywhere else.

We then assign labels to each measurement start and end state to distinguish starts from ends, which measurement that start/end belongs to, and which instrument that measurement belongs to. Then, the only change to the Kalman filter is to reset each z_i state *only* at the start of an exposure belonging to that instrument. That way, z_i is allowed to accumulate simultaneously with other instruments, thereby building in all cross correlations during overlaps by construction. Likewise, the RTS smoother must undo the reset for *only* that instrument. That is, for instrument i and integral state index j , the reset matrix is a block-diagonal matrix which is the identity except for the z_j component matching the instrument,

$$\mathbf{R}_i = \begin{pmatrix} \mathbf{I} & & \\ & 1 - \delta_{i0} & \\ & & 1 - \delta_{i1} \\ & & & \ddots \end{pmatrix}, \quad \delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}. \quad (3.18)$$

Then, exposure-end states in the Kalman filter project only that instrument's z_i .

Finally, the initial covariance $\tilde{\mathbf{P}}_0$ is undefined for the integral state. We choose to initialize to the identity for each integral state, though in practice it will not matter because these elements will be immediately reset upon

reaching the first exposure-start state for that instrument. The identity is convenient because it makes P_0 invertible, which is needed for retrodiction (smoothing from the first exposure-start state),

$$P_0 = \begin{pmatrix} P_\infty & & \\ & 1 & \\ & & \ddots \end{pmatrix}. \quad (3.19)$$

With these adjustments to the augmented model, and labels identifying the measurement and instrument at each exposure start and end state, we can simply iterate through the $2N$ states chronologically using the integrated Kalman and RTS prescriptions defined in the previous sections, resetting integral states at their corresponding instrument's exposure-start states and reading them off at the corresponding exposure-end states.

3.2.4. Parallel Implementation

The sequential Kalman and RTS algorithms discussed so far iterate once through the K states in a forward pass and again in reverse. $K = N$ in the instantaneous case, and $K = 2N$ in the integrated case; hence, the time complexity is $\mathcal{O}(N)$.

Särkkä & García-Fernández (2021) (hereafter SG21) showed that, because of the associativity of the filtering ($p(x_k|y_{1:k})$) and smoothing ($p(x_k|y_{1:k}, x_{k+1})$) distributions¹⁵, the sequential Kalman filter and RTS smoother can be reframed as an all-prefix-sums problem (e.g., Blelloch 1990), which can be solved using a parallel-scan algorithm on a GPU. The parallel method reduces the wall-clock time complexity to $\mathcal{O}(N/T + \log T)$, for T parallel workers. If $T \sim N$, the optimal scaling of $\mathcal{O}(\log N)$ is achieved. For large datasets, usually $T \ll N$, giving $\mathcal{O}(N/T)$. An extensive experimental evaluation of these algorithms on GPUs can be found in Särkkä & García-Fernández (2025), who also present a method to compute the parallel Kalman and RTS simultaneously on two GPUs. Nonlinear and square-root extensions of these methods can be found in Yaghoobi et al. (2021); Yaghoobi et al. (2025).

Recently, Yaghoobi & Särkkä (2025), based on the original method of SG21, implemented a parallel Kalman filter and RTS smoother for an integral observation model using fast and slow-rate states. In our case, we have the integral state augmented directly into the modeled state vector. As such, we can use the same formalism as SG21 to reframe Eq. 3.15 and 3.16 in terms of their associative parts. Here, we summarize the neces-

sary changes to make the SG21 method compatible with integrated measurements.

Similar to the integrated sequential Kalman filter and RTS smoother, we handle the start and end states separately. For the parallel Kalman filter, the necessary associative parameters to calculate upfront for each of the K steps are $(\mathbf{A}_k, \mathbf{b}_k, \mathbf{C}_k, \boldsymbol{\eta}_k, \mathbf{J}_k)$; note that these are defined as in SG21 (i.e., $\mathbf{A}_k$ is *not* the transition matrix $\mathbf{A}_k$ as used previously in this manuscript). The 5-tuple of associative parameters for a transition to an exposure-end state k are given exactly by Eq. 10 of SG21. For transitions to an exposure-start state, the reset requires us to set $\mathbf{F}_{\text{eff},k-1} = \mathbf{R} \mathbf{F}_{k-1}$ (in the associative notation, $\mathbf{F}_{k-1}$ is the transition matrix) and $\mathbf{Q}_{\text{eff},k-1} = \mathbf{R} \mathbf{Q}_{k-1} \mathbf{R}^T$. The lack of a measurement at k requires us to set $R_k \leftarrow \infty$. The 5-tuple at an exposure-start state is then

$$\begin{aligned} \mathbf{A}_k &= \mathbf{F}_{\text{eff},k-1}, \\ \mathbf{b}_k &= \mathbf{0}, \\ \mathbf{C}_k &= \mathbf{Q}_{\text{eff},k-1}, \\ \boldsymbol{\eta}_k &= \mathbf{0}, \\ \mathbf{J}_k &= \mathbf{0}. \end{aligned} \quad (3.20)$$

The initial $k = 1$ step (Eq. 11 in SG21) is also modified because it is always an exposure-start state for our problem formulation,

$$\begin{aligned} \mathbf{m}_1^- &= \mathbf{F}_{\text{eff},0} \mathbf{m}_0, \\ \mathbf{P}_1^- &= \mathbf{F}_{\text{eff},0} \mathbf{P}_0 \mathbf{F}_{\text{eff},0}^T + \mathbf{Q}_0, \\ \mathbf{A}_1 &= \mathbf{R}, \\ \mathbf{b}_1 &= \mathbf{m}_1^-, \\ \mathbf{C}_1 &= \mathbf{P}_1^-, \\ \boldsymbol{\eta}_1 &= \mathbf{0}, \\ \mathbf{J}_1 &= \mathbf{0}. \end{aligned} \quad (3.21)$$

For the parallel RTS smoother, we require a 3-tuple of associative parameters $(\mathbf{E}_k, \mathbf{g}_k, \mathbf{L}_k)$ for each of the K steps. The 3-tuple at the final $k = K$ step (an exposure-end state) are unchanged from SG21:

$$\begin{aligned} \mathbf{E}_K &= \mathbf{0}, \\ \mathbf{g}_K &= \mathbf{m}_K, \\ \mathbf{L}_K &= \mathbf{P}_K. \end{aligned} \quad (3.22)$$

Likewise, the 3-tuple when k is an exposure-end state is also the same as in SG21. Exposure-start states are modified to match the behavior of Eq. 3.16. Namely, they require the quantities to smooth be the predicted pre-reset state $(\mathbf{m}_k^-, \mathbf{P}_k^-)$ and that $\mathbf{F}_{\text{eff},k} = \mathbf{F}_k \mathbf{R}$, giving

$$\begin{aligned} \mathbf{E}_k &= \mathbf{P}_k^- \mathbf{F}_{\text{eff},k}^T (\mathbf{F}_{\text{eff},k} \mathbf{P}_k^- \mathbf{F}_{\text{eff},k}^T + \mathbf{Q}_k)^{-1}, \\ \mathbf{g}_k &= \mathbf{m}_k^- - \mathbf{E}_k \mathbf{F}_{\text{eff},k} \mathbf{m}_k^-, \\ \mathbf{L}_k &= \mathbf{P}_k^- - \mathbf{E}_k \mathbf{F}_{\text{eff},k} \mathbf{P}_k^-. \end{aligned} \quad (3.23)$$

¹⁵ And, consequently, the marginal likelihoods $p(y_{1:k})$.

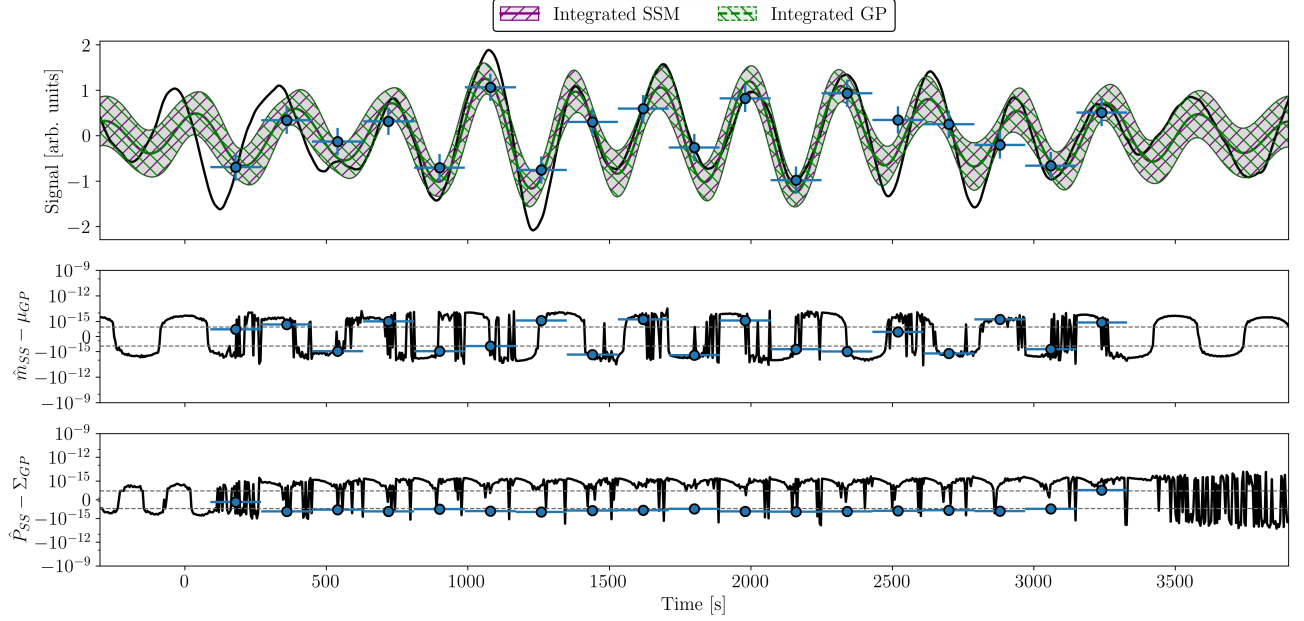


Figure 2. Numerical validation that the integrated SSM (purple curve) presented in this work produces the same result as the full integrated GP (green curve) from L26. **Top:** An example stochastic signal (black curve) from an SHO kernel with a ~ 300 sec timescale (Appendix B). The data points are mock measurements of this curve with 180 s exposures and uncertainties of 0.3 (arb. units). **Middle:** The difference between the conditioned (points) and predicted (black curve) mean of the SSM and GP approach. **Bottom:** The same for the variances. The horizontal dashed lines show machine epsilon for a single 64-bit floating-point numbers in the dynamic range of the simulated data; this also corresponds to the linear regime of the y-axis symmetric log scale. The two methods generally agree to within an order of magnitude of this level, consistent with the accumulated floating-point error over all the flops in the calculation. Periodicity in the residuals likely stem from the same periodicity in the kernel propagating through the various computations that yield the final mean and variance.

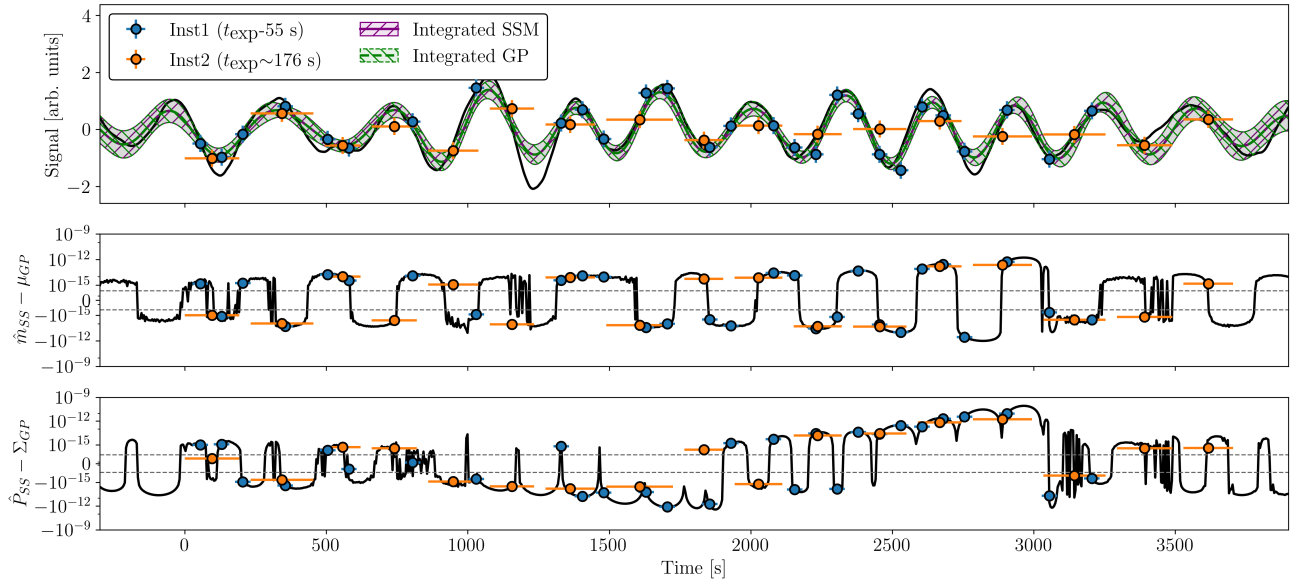


Figure 3. The same as Figure 2 but for a mock dataset with two instruments; one with 55 s exposures and one with variable 180 ± 30 s, which frequently overlap. Again, the two methods generally agree to within the expectation from numerical precision. Numerical stability in the variance may be improved with square-root filters (e.g., Yaghoobi et al. 2025).

The K 5-tuples for Kalman filtering (Eqs. 3.20 and 3.21 for exposure starts, Eq. 10 of SG21 for exposure ends) and the K 3-tuples for RTS smoothing (Eqs. 3.22 and 3.23 for exposure starts, SG21 for exposure ends) are each combined via the appropriate binary associative operator (see SG21). We implemented this method in the `ParallelIntegratedStateSpaceSolver` class of `smolgp` using the JAX parallel scan function `jax.lax.associative_scan`.

4. VALIDATION AND PERFORMANCE

First, we numerically validated the equivalence between our integrated SSM and the integrated GP framework of L26. For the former, we used the `IntegratedStateSpaceModel` class of `smolgp`, and for the latter we defined a custom `tinygp` kernel which implements the various cases and (sub)integrals defined in L26 for all pairs of integrated measurements. We generated a single and multi-instrument synthetic dataset by sampling from a “true” latent signal, defined by an SHO GP. We then computed the conditioned mean and variance at the data points (integrated over the exposure interval) as well as the predicted latent (instantaneous) curve at a dense grid of test points.

Figure 2 shows a representative example with a single instrument for the edge case of no deadtime between exposures. Both the integrated SSM and GP methods agree at the level of machine precision when working with 64-bit floating-point numbers.

A multi-instrument example is shown in Figure 3 which has instrument 1 taking constant 55 s exposures (similar to NEID solar data, Lin et al. 2022) and instrument 2 taking variable exposure times sampled from a Gaussian with mean 180 s and standard deviation 30 s (similar to EXPRES solar data, Llama et al. 2024). In this case, the two methods generally agreed at the level expected from numerical precision when working with 64-bit floating-point numbers. Likewise, the likelihoods computed by all methods agreed to within 10^{-14} . We also verified that the parallel method gives the same result as the sequential solver in all cases.

Finally, we benchmarked the full GP, QSM, and SSM methodologies to compare performance. We only show the performance for the full GP, QSM, and sequential SSM solvers as tested on a CPU (Intel® Xeon® w5-3435X) because their performance was degraded on a GPU; conversely, we only show the parallel SSM solver as tested on a GPU (NVIDIA RTX 6000 Ada, CUDA v12.8) because its CPU performance was degraded. Our setup was as follows. For each function to benchmark (likelihood, conditioning, and prediction), we set up `tinygp` objects for the full and QSM representations of

an SHO kernel and a `smolgp` object for the SSM representation. We took a random draw from the prior distribution as our true signal, from which we generated random measurements of varying size N . We timed five computations of each function at each N , while monitoring their peak memory usage in a separate thread; this peak memory use does not include overheads before calling the function (e.g. ~ 200 MB from importing JAX and JIT compilation).

The average and standard deviation of the five runs, for both the runtime and the peak memory use, are shown in Figure 4 for the tests involving instantaneous data and Figure 5 for those involving integrated data. The most dramatic improvements are seen with integrated data, since there is no QSM representation to compete with the SSM method. Generally, we found the SSM method to be the most memory efficient, which can easily become limiting at modest $N \sim 10^5$ with the full GP method ($\gtrsim 100$ GB of RAM), as well as for predictions with the QSM method. As tested, the SSM method is the fastest for making predictions, and when parallelized on a GPU is the fastest at conditioning and rivals QSM methods at computing the likelihood.

5. SUMMARY AND DISCUSSION

We have developed a state-space framework for solving the general GP regression problem when the data contain integrated, possibly overlapping, measurements. Compared to the method of L26 which constructs the full, dense covariance matrix and integrates the covariance function between all pairs of data points (accounting for overlap), our method yields equivalent results but can be solved in linear (and even logarithmic) time complexity. This is achieved by augmenting a linear Gaussian SSM with an integral state z such that $dz/dt = x$, which we reset to zero at the start of the exposures and make noisy measurements of (normalized by the exposure length) at the ends of the exposures.

We derived the matching Kalman filter and RTS smoother (Section 3) and demonstrated numerical equivalence to L26 (Section 4) to near machine precision, though lingering structured residuals likely stem from some numerical instabilities in the filtering/smoothing algorithms; these may be remedied with square-root filters, which can also be parallelized (Yaghoobi et al. 2025). We also presented a speed and memory optimal method, Algorithm 1, for predicting at arbitrary points once the conditioned means and covariances at all the states have been obtained. Overall, the SSM framework, especially its compatibility with parallel methods (Section 3.2.4), brings integrated and multi-instrument datasets into the same (or faster) regime as state-of-the-

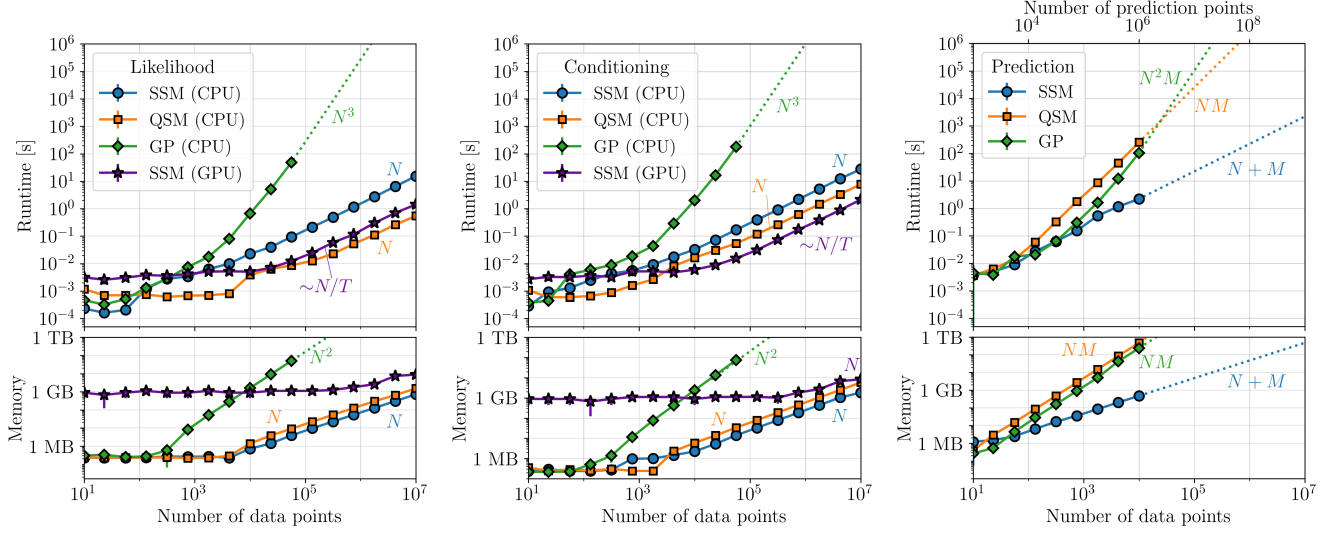


Figure 4. Runtime and memory benchmarking comparing the performance on instantaneous data of the full/dense GP solution (i.e. Eq. 2.4, as in `tinygp`, green diamonds), using QSM algebra (also via `tinygp`, orange squares), and our implementation of the sequential SSM solver (`smolgp`, blue circles); these three cases were all tested on a CPU because their performance on a GPU was degraded. Dashed lines trace the theoretical scaling from the largest value tested. In all cases, the top panel shows the wall-clock timed average of five runs, while the bottom shows the peak memory usage during the function execution. The purple stars show the parallel SSM solver (SG21 as implemented in `smolgp`) as tested on a NVIDIA RTX 6000 Ada GPU running CUDA v12.8. **Left:** Results for the log-likelihood as a function of N data points. **Middle:** Results for conditioning at the N data points, including initialization. **Right:** Results for conditioning on N data points and then predicting at $M = 100N$ test points, to simulate a typical high-resolution prediction scenario. **Takeaway:** The SSM shares the linear runtime scaling as QSMs but is typically more memory efficient (especially for predictions). Computing the likelihood is faster in the QSM framework, although for conditioning, the best runtime performance is achieved by the parallel SSM (with high memory overhead on a GPU).

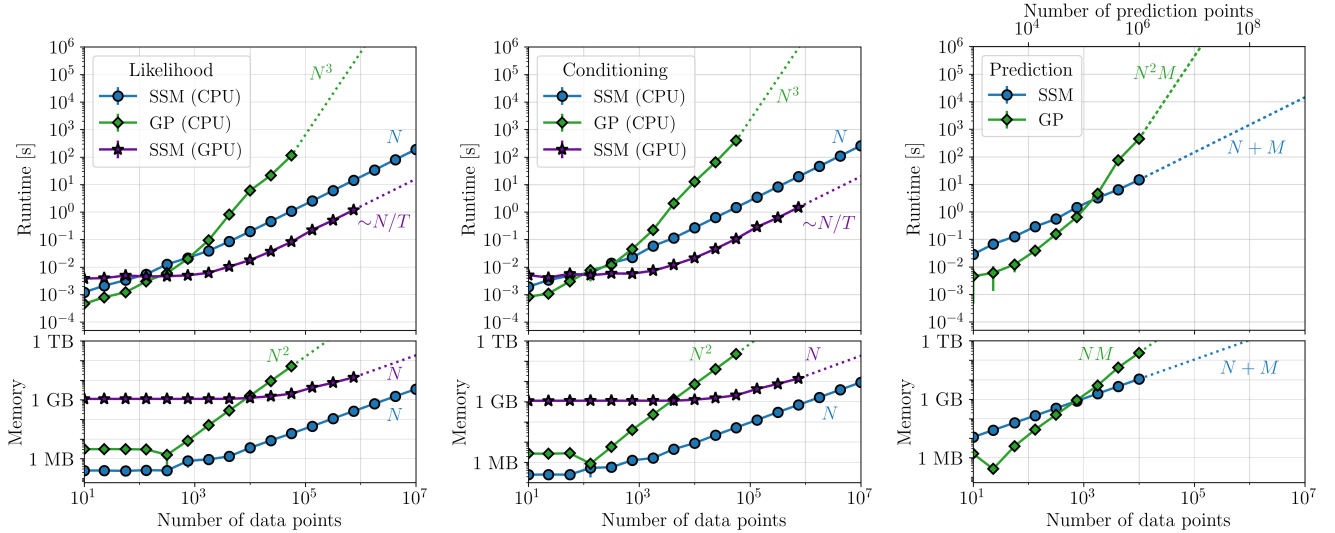


Figure 5. Same as Figure 4 but with integrated measurements. The full/dense GP solution (green diamonds) uses the L26 method, which we implemented in `tinygp`. The SSM solution (blue circles) uses our augmented SSM approach (Section 3), as implemented in `smolgp`. Likewise, the parallel version (purple circles) implements Section 3.2.4 in `smolgp`. **Takeaway:** Because there is no quasi-separable framework compatible with integrated measurements, the SSM method provides a transformative speedup and memory savings over traditional GP methods which are forced to construct the full covariance matrix.

art QSM methods, opening the door to holistic analyses of massive multi-instrument datasets ($N > 10^5$).

The integrated SSM-GP approach discussed here is not just limited to time series, but any binned data where the independent coordinate is sortable (i.e. there must be a unique causal sequence for the Kalman/RTS scans). Likewise, multivariate data (i.e. parallel time series) are also compatible with an appropriately shaped observation matrix; the example used to demonstrate the parallel method in SG21 is a 2D trajectory through space. SSMs can also handle spatiotemporal models (Särkkä & Hartikainen 2012; Särkkä et al. 2013), non-linear models (Frigola et al. 2014), non-Gaussian/non-conjugate likelihoods (Chang et al. 2020), certain non-stationary kernels (Benavoli & Zaffalon 2016; Lin et al. 2025), model learning problems (Eleftheriadis et al. 2017), and even time-variable hyperparameters (via inducing points, Liu & Djurić 2021). These flexible extensions of SSMs could provide significant utility to other, more complex astronomical modeling problems which demand scalable methods. Additionally, because the modeled latent state contains not just the instantaneous state but also its derivatives, SSMs are a natural, powerful choice for modeling phenomena that include derivative observations; one example is the FF’ model

(Aigrain et al. 2012; Rajpaul et al. 2015; Jones et al. 2022; Gilbertson et al. 2020; Tran et al. 2023).

We implemented the full SSM-GP framework discussed throughout this manuscript in the Python/JAX package `smolgp`, which is available (under the MIT license) at <https://github.com/smolgp-dev/smolgp>. The `smolgp` design philosophy is to have as identical an API as possible to `tinygp` (Foreman-Mackey et al. 2024) so existing analyses can easily incorporate these new features. We encourage contributions and engagement from the community on how best to expand these methods to other astronomical problems of interest.

ACKNOWLEDGMENTS

The authors gratefully thank the anonymous referee for their thoughtful feedback which greatly improved clarity throughout the manuscript. We also thank Lehman Garrison, Joseph Long, David Hogg, Adrian Price-Whelan, and the CCA Astro Data and Software groups for engaging and helpful discussions on methodology, code design, algorithms, and statistics. R.A.R. was supported by the Flatiron Research Fellowship at the Flatiron Institute, a division of the Simons Foundation.

Software: `smolgp` (Rubenzahl & Hattori 2026), `tinygp` (Foreman-Mackey et al. 2024), JAX (Bradbury et al. 2018), `matplotlib` (Hunter 2007)

REFERENCES

- Aigrain, S., & Foreman-Mackey, D. 2023, *ARA&A*, 61, 329, doi: [10.1146/annurev-astro-052920-103508](https://doi.org/10.1146/annurev-astro-052920-103508)
- Aigrain, S., Pont, F., & Zucker, S. 2012, *MNRAS*, 419, 3147, doi: [10.1111/j.1365-2966.2011.19960.x](https://doi.org/10.1111/j.1365-2966.2011.19960.x)
- Ambikasaran, S., Foreman-Mackey, D., Greengard, L., Hogg, D. W., & O’Neil, M. 2015, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 38, 252, doi: [10.1109/TPAMI.2015.2448083](https://doi.org/10.1109/TPAMI.2015.2448083)
- Anderson, B. D. O., & Moore, J. B. 1979, *Optimal Filtering* (Prentice-Hall)
- Benavoli, A., & Zaffalon, M. 2016, arXiv e-prints, arXiv:1601.01544, doi: [10.48550/arXiv.1601.01544](https://doi.org/10.48550/arXiv.1601.01544)
- Blelloch, G. E. 1990, *Prefix Sums and Their Applications*, Tech. Rep. CMU-CS-90-190, School of Computer Science, Carnegie Mellon University, doi: [10.1184/R1/6608579.V1](https://doi.org/10.1184/R1/6608579.V1)
- Bradbury, J., Frostig, R., Hawkins, P., et al. 2018, JAX: composable transformations of Python+NumPy programs, 0.3.13. <http://github.com/jax-ml/jax>
- Brown, T. M., Gilliland, R. L., Noyes, R. W., & Ramsey, L. W. 1991, *ApJ*, 368, 599, doi: [10.1086/169725](https://doi.org/10.1086/169725)
- Chang, P. E., Wilkinson, W. J., Khan, M. E., & Solin, A. 2020, in *2020 IEEE 30th International Workshop on Machine Learning for Signal Processing (MLSP)*, 1–6, doi: [10.1109/MLSP49062.2020.9231560](https://doi.org/10.1109/MLSP49062.2020.9231560)
- Chaplin, W. J., Cegla, H. M., Watson, C. A., Davies, G. R., & Ball, W. H. 2019, *AJ*, 157, 163, doi: [10.3847/1538-3881/ab0c01](https://doi.org/10.3847/1538-3881/ab0c01)
- Chen, C.-T. 1999, *Linear System Theory and Design* (3rd Edition) (Oxford University Press). <https://app.knovel.com/hotlink/toc/id:kpLSTDE003/linear-system-theory/linear-system-theory>
- Crass, J., Gaudi, B. S., Leifer, S., et al. 2021, arXiv e-prints, arXiv:2107.14291, doi: [10.48550/arXiv.2107.14291](https://doi.org/10.48550/arXiv.2107.14291)
- Delisle, J. B., Hara, N., & Ségransan, D. 2020, *A&A*, 638, A95, doi: [10.1051/0004-6361/201936906](https://doi.org/10.1051/0004-6361/201936906)
- Delisle, J. B., Unger, N., Hara, N. C., & Ségransan, D. 2022, *A&A*, 659, A182, doi: [10.1051/0004-6361/202141949](https://doi.org/10.1051/0004-6361/202141949)
- Dumusque, X., Udry, S., Lovis, C., Santos, N. C., & Monteiro, M. J. P. F. G. 2011, *A&A*, 525, A140, doi: [10.1051/0004-6361/201014097](https://doi.org/10.1051/0004-6361/201014097)

- Duvenaud, D. 2014, PhD thesis, University of Cambridge, doi: [10.17863/CAM.14087](https://doi.org/10.17863/CAM.14087)
- Eidelman, Y., & Gohberg, I. 1999, *Integral equations and operator theory*, 34, 293, doi: [10.1007/BF01300581](https://doi.org/10.1007/BF01300581)
- Eleftheriadis, S., Nicholson, T. F. W., Deisenroth, M. P., & Hensman, J. 2017, arXiv e-prints, arXiv:1705.10888, doi: [10.48550/arXiv.1705.10888](https://doi.org/10.48550/arXiv.1705.10888)
- Fatehi, A., & Huang, B. 2017, *IEEE Transactions on Instrumentation and Measurement*, 66, 2490, doi: [10.1109/TIM.2017.2701143](https://doi.org/10.1109/TIM.2017.2701143)
- Flower, J. O. 1993, in *Electrical Engineer's Reference Book (Fifteenth Edition)*, fifteenth edition edn., ed. G. R. Jones, M. A. Laughton, & M. G. Say (Oxford: Newnes), 5/1–5/35, doi: [10.1016/B978-0-7506-1202-9.50009-2](https://doi.org/10.1016/B978-0-7506-1202-9.50009-2)
- Foreman-Mackey, D., Agol, E., Ambikasaran, S., & Angus, R. 2017, *The Astronomical Journal*, 154, 220, doi: [10.3847/1538-3881/aa9332](https://doi.org/10.3847/1538-3881/aa9332)
- Foreman-Mackey, D., Yu, W., Yadav, S., et al. 2024, dfm/tinygp: The tiniest of Gaussian Process libraries, v0.3.0, Zenodo, doi: [10.5281/zenodo.10463641](https://doi.org/10.5281/zenodo.10463641)
- Frigola, R., Chen, Y., & Rasmussen, C. E. 2014, in *Advances in Neural Information Processing Systems*, ed. Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, & K. Weinberger, Vol. 27 (Curran Associates, Inc.). https://proceedings.neurips.cc/paper_files/paper/2014/file/875b60cf697a9e09e12c0f07b982e431-Paper.pdf
- Gilbertson, C., Ford, E. B., Jones, D. E., & Stenning, D. C. 2020, *ApJ*, 905, 155, doi: [10.3847/1538-4357/abc627](https://doi.org/10.3847/1538-4357/abc627)
- Gordon, T. A., Agol, E., & Foreman-Mackey, D. 2020, *AJ*, 160, 240, doi: [10.3847/1538-3881/abbc16](https://doi.org/10.3847/1538-3881/abbc16)
- Guo, Y., & Huang, B. 2015, *Automatica*, 58, 32, doi: <https://doi.org/10.1016/j.automatica.2015.05.001>
- Hartikainen, J., & Särkkä, S. 2010, in *2010 IEEE International Workshop on Machine Learning for Signal Processing*, 379–384, doi: [10.1109/MLSP.2010.5589113](https://doi.org/10.1109/MLSP.2010.5589113)
- Hu, Z., & Tak, H. 2020, *AJ*, 160, 265, doi: [10.3847/1538-3881/abc1e2](https://doi.org/10.3847/1538-3881/abc1e2)
- Huber, D., Bedding, T. R., Stello, D., et al. 2011, *ApJ*, 743, 143, doi: [10.1088/0004-637X/743/2/143](https://doi.org/10.1088/0004-637X/743/2/143)
- Hunter, J. D. 2007, *CSE*, 9, 90
- Jones, D. E., Stenning, D. C., Ford, E. B., et al. 2022, *The Annals of Applied Statistics*, 16, 652, doi: [10.1214/21-AOAS1471](https://doi.org/10.1214/21-AOAS1471)
- Jordán, A., Eyheramendy, S., & Buchner, J. 2021, *Research Notes of the American Astronomical Society*, 5, 107, doi: [10.3847/2515-5172/abfe68](https://doi.org/10.3847/2515-5172/abfe68)
- Kalman, R. E. 1960, *Journal of Basic Engineering*, 82, 35, doi: [10.1115/1.3662552](https://doi.org/10.1115/1.3662552)
- Kelly, B. C., Becker, A. C., Sobolewska, M., Siemiginowska, A., & Uttley, P. 2014, *The Astrophysical Journal*, 788, 33, doi: [10.1088/0004-637X/788/1/33](https://doi.org/10.1088/0004-637X/788/1/33)
- Kimpson, T., O'Neill, N. J., Meyers, P. M., & Melatos, A. 2025, arXiv e-prints, arXiv:2510.11077, doi: [10.48550/arXiv.2510.11077](https://doi.org/10.48550/arXiv.2510.11077)
- Kjeldsen, H., & Bedding, T. R. 1995, *A&A*, 293, 87, doi: [10.48550/arXiv.astro-ph/9403015](https://doi.org/10.48550/arXiv.astro-ph/9403015)
- Lin, A. S. J., Monson, A., Mahadevan, S., et al. 2022, *AJ*, 163, 184, doi: [10.3847/1538-3881/ac5622](https://doi.org/10.3847/1538-3881/ac5622)
- Lin, Z., Li, Y., Yin, F., Maroñas, J., & Thiéry, A. H. 2025, *IEEE Transactions on Signal Processing*, 73, 5229, doi: [10.1109/TSP.2025.3643309](https://doi.org/10.1109/TSP.2025.3643309)
- Liu, Y., & Djurić, P. M. 2021, *Proceedings of the European Signal Processing Conference*, 1462, doi: [10.23919/Eusipco47968.2020.9287481](https://doi.org/10.23919/Eusipco47968.2020.9287481)
- Llama, J., Zhao, L. L., Brewer, J. M., et al. 2024, in *Society of Photo-Optical Instrumentation Engineers (SPIE) Conference Series*, Vol. 13094, Ground-based and Airborne Telescopes X, ed. H. K. Marshall, J. Spyromilio, & T. Usuda, 130942O, doi: [10.1117/12.3020494](https://doi.org/10.1117/12.3020494)
- Loper, J., Blei, D., Cunningham, J. P., & Paninski, L. 2021, *Journal of Machine Learning Research*, 22, 1. <http://jmlr.org/papers/v22/21-0072.html>
- Luhn, J. K., Rubenzahl, R. A., Halverson, S., & Zhao, L. L. 2026, *AJ*, 171, 342, doi: [10.3847/1538-3881/ae5d38](https://doi.org/10.3847/1538-3881/ae5d38)
- Miller, A. C., Anderson, L., Leistedt, B., et al. 2022, *Ann. Appl. Stat.*, 16, 2672, doi: [10.1214/22-AOAS1608](https://doi.org/10.1214/22-AOAS1608)
- Pernet, C., Signargout, H., & Villard, G. 2023, in *Proceedings of the 2023 International Symposium on Symbolic and Algebraic Computation, ISSAC '23* (New York, NY, USA: Association for Computing Machinery), 480–489, doi: [10.1145/3597066.3597102](https://doi.org/10.1145/3597066.3597102)
- Pernet, C., & Storjohann, A. 2018, *Journal of Symbolic Computation*, 85, 224, doi: [10.1016/j.jsc.2017.07.010](https://doi.org/10.1016/j.jsc.2017.07.010)
- Phillips, D. F., Glenday, A. G., Dumusque, X., et al. 2016, in *SPIE, Vol. 9912, Advances in Optical and Mechanical Technologies for Telescopes and Instrumentation II*, ed. R. Navarro & J. H. Burge, 99126Z, doi: [10.1117/12.2232452](https://doi.org/10.1117/12.2232452)
- Qian, N., & Chang, G. 2021, *Measurement*, 176, 109209, doi: <https://doi.org/10.1016/j.measurement.2021.109209>
- Rajpaul, V., Aigrain, S., Osborne, M. A., Reece, S., & Roberts, S. 2015, *MNRAS*, 452, 2269, doi: [10.1093/mnras/stv1428](https://doi.org/10.1093/mnras/stv1428)
- Rasmussen, C. E., & Williams, C. K. I. 2006, *Gaussian Processes for Machine Learning* (The MIT Press)
- Rauch, H. E., Tung, F., & Striebel, C. T. 1965, *AIAA Journal*, 3, 1445, doi: [10.2514/3.3166](https://doi.org/10.2514/3.3166)

- Rubenzahl, R., & Hattori, S. 2026, smolgp-dev/smolgp: Initial Release, v0.1.0, Zenodo, doi: [10.5281/zenodo.18418838](https://doi.org/10.5281/zenodo.18418838)
- Rubenzahl, R. A., Halverson, S., Walawender, J., et al. 2023, Publications of the Astronomical Society of the Pacific, 135, 125002, doi: [10.1088/1538-3873/ad0b30](https://doi.org/10.1088/1538-3873/ad0b30)
- Salehi, Y., Fatehi, A., & Nayebi, M. 2018, IEEE Transactions on Instrumentation and Measurement, PP, 1, doi: [10.1109/TIM.2018.2884604](https://doi.org/10.1109/TIM.2018.2884604)
- Särkkä, S., & García-Fernández, Á. F. 2025, arXiv e-prints, arXiv:2511.10363, doi: [10.48550/arXiv.2511.10363](https://doi.org/10.48550/arXiv.2511.10363)
- Särkkä, S., & García-Fernández, n. F. 2021, IEEE Transactions on Automatic Control, 66, 299, doi: [10.1109/TAC.2020.2976316](https://doi.org/10.1109/TAC.2020.2976316)
- Särkkä, S., & Hartikainen, J. 2012, in Proceedings of Machine Learning Research, Vol. 22, Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics, ed. N. D. Lawrence & M. Girolami (La Palma, Canary Islands: PMLR), 993–1001. <https://proceedings.mlr.press/v22/sarkka12.html>
- Särkkä, S., & Solin, A. 2019, Applied Stochastic Differential Equations (Cambridge University Press). <https://users.aalto.fi/~asolin/sde-book/sde-book.pdf>
- Särkkä, S., Solin, A., & Hartikainen, J. 2013, IEEE Signal Processing Magazine, 30, 51, doi: [10.1109/MSP.2013.2246292](https://doi.org/10.1109/MSP.2013.2246292)
- Särkkä, S., & Svensson, L. 2023, Bayesian Filtering and Smoothing (Cambridge University Press), doi: [10.1017/9781108917407](https://doi.org/10.1017/9781108917407)
- Solin, A., & Särkkä, S. 2014a, in Proceedings of Machine Learning Research, Vol. 33, Proceedings of the Seventeenth International Conference on Artificial Intelligence and Statistics, ed. S. Kaski & J. Corander (Reykjavik, Iceland: PMLR), 904–912. <https://proceedings.mlr.press/v33/solin14.html>
- Solin, A., & Särkkä, S. 2014b, in 2014 IEEE International Workshop on Machine Learning for Signal Processing (MLSP), 1–6, doi: [10.1109/MLSP.2014.6958899](https://doi.org/10.1109/MLSP.2014.6958899)
- Tran, Q. H., Bedell, M., Foreman-Mackey, D., & Luger, R. 2023, ApJ, 950, 162, doi: [10.3847/1538-4357/acd05c](https://doi.org/10.3847/1538-4357/acd05c)
- Van Loan, C. 1978, IEEE Transactions on Automatic Control, 23, 395, doi: [10.1109/TAC.1978.1101743](https://doi.org/10.1109/TAC.1978.1101743)
- Wilkinson, W. J., Särkkä, S., & Solin, A. 2023, Journal of Machine Learning Research, 24, 1. <http://jmlr.org/papers/v24/21-1298.html>
- Yaghoobi, F., Corenflos, A., Hassan, S., & Särkkä, S. 2021, in ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 5350–5354, doi: [10.1109/ICASSP39728.2021.9413364](https://doi.org/10.1109/ICASSP39728.2021.9413364)
- Yaghoobi, F., Corenflos, A., Hassan, S., & Särkkä, S. 2025, SIAM Journal on Scientific Computing, 47, B454, doi: [10.1137/23M156121X](https://doi.org/10.1137/23M156121X)
- Yaghoobi, F., & Särkkä, S. 2025, IEEE Signal Processing Letters, 32, 371, doi: [10.1109/LSP.2024.3519258](https://doi.org/10.1109/LSP.2024.3519258)

APPENDIX

A. EXPOSURE TIME INTEGRATION FOR CELERITE MODELS

Here, we give expressions for the exposure time integrated versions of the *celerite* kernel and its power spectrum. It turns out that the time-integrated *celerite* kernel remains expressible as a semiseparable matrix when the exposures do not overlap, but when exposures overlap for nonzero time lags, the kernel no longer is semiseparable.

For the *celerite* model, the general kernel function is expressed as (Foreman-Mackey et al. 2017)

$$k(\Delta) = ae^{-c\tau} \cos(d\Delta) + be^{-c\tau} \sin(d\Delta), \quad (\text{A1})$$

where $\Delta = |t_i - t_j|$ is the lag between times t_i and t_j , and the parameters of the model are a , b , c , and d . If the two measurements have exposure time δ , then the kernel becomes the following integral (using the notation of L26),

$$k_{FF}(\delta, \Delta) = \frac{1}{\delta^2} \int_{t_i - \frac{\delta}{2}}^{t_i + \frac{\delta}{2}} \int_{t_j - \frac{\delta}{2}}^{t_j + \frac{\delta}{2}} k(|t - t'|) dt dt'. \quad (\text{A2})$$

When $\delta \leq \Delta$, the exposures do not overlap and the integrated model still falls within the class of *celerite* models

$$k_{FF}(\delta, \Delta | \delta \leq \Delta) = Ae^{-c\Delta} \cos(d\Delta) + Be^{-c\Delta} \sin(d\Delta), \quad (\text{A3})$$

where c and d have not changed, but the amplitudes are given by the updated expressions

$$A = \frac{2C_1 [\cosh(c\delta) \cos(d\delta) - 1] - 2C_2 \sinh(c\delta) \sin(d\delta)}{\delta^2(c^2 + d^2)^2}, \quad (\text{A4})$$

and

$$B = \frac{2C_2 [\cosh(c\delta) \cos(d\delta) - 1] + 2C_1 \sinh(c\delta) \sin(d\delta)}{\delta^2(c^2 + d^2)^2}, \quad (\text{A5})$$

with

$$C_1 = ac^2 - ad^2 + 2bcd \quad \text{and} \quad C_2 = bc^2 - bd^2 - 2acd. \quad (\text{A6})$$

For the case of overlapping exposures ($0 \leq \Delta < \delta$), the result is not as simple because of the absolute value in the definition of Δ . In this case, we take $t_i > t_j$ without loss of generality, and separate the integral into three integrals that can be easily evaluated

$$k_{FF}(\delta, \Delta | \Delta < \delta) = \frac{1}{\delta^2} \left(\int_{t_j + \frac{\delta}{2}}^{t_i + \frac{\delta}{2}} \int_{t_j - \frac{\delta}{2}}^{t_j + \frac{\delta}{2}} k(t - t') dt' dt + \int_{t_i - \frac{\delta}{2}}^{t_j + \frac{\delta}{2}} \int_{t_j - \frac{\delta}{2}}^t k(t - t') dt' dt + \int_{t_i - \frac{\delta}{2}}^{t_j + \frac{\delta}{2}} \int_t^{t_j + \frac{\delta}{2}} k(t' - t) dt' dt \right). \quad (\text{A7})$$

Evaluating and simplifying the result gives

$$\begin{aligned} k_{FF}(\delta, \Delta | \Delta < \delta) &= k_{FF}(\delta, \Delta | \delta \leq \Delta) + 2 [(ac + bd)(c^2 + d^2)(\delta - \Delta) \\ &\quad - C_1 \sinh(c\delta - c\Delta) \cos(d\delta - d\Delta) \\ &\quad + C_2 \cosh(c\delta - c\Delta) \sin(d\delta - d\Delta)] / [\delta(c^2 + d^2)]^2, \end{aligned} \quad (\text{A8})$$

where the first term is the result from Eq. A3. One can see that this cannot be expressed as a *celerite* term.

The power spectrum of the time-integrated term derived above can be computed by taking the Fourier transform and, after some simplifications, it reduces to the expected result

$$S(\delta, \omega) = S(\omega) \text{sinc}^2(\omega\delta/2), \quad (\text{A9})$$

where $S(\omega)$ is the power spectrum of the original term and $\text{sinc}(x) = \sin(x)/x$ is the Fourier transform of a top hat representing the exposure. An interesting consequence of this form is that, since the sinc function goes to zero when $\omega\delta = 2\pi n$ for integers $n \neq 0$, there is no power at integer multiples of the sampling frequency $f = \frac{2\pi}{\omega} = \delta^{-1}$.

An important implementation note is that, while a nonintegrated *celerite* term would generate a matrix with the amplitude a on the diagonal, the diagonal for the integrated term is not simply A because the lag on the diagonal is $\Delta = 0 < \delta$. This means that the diagonal entries in the covariance matrix for a time-integrated *celerite* model are

$$k_{FF}(\delta, \Delta = 0) = A + \frac{2[(ac + bd)(c^2 + d^2)\delta - C_1 \sinh(c\delta) \cos(d\delta) + C_2 \cosh(c\delta) \sin(d\delta)]}{\delta^2(c^2 + d^2)^2} \quad (\text{A10})$$

which can be simplified to

$$k_{FF}(\delta, \Delta = 0) = \frac{2[(ac + bd)(c^2 + d^2)\delta - C_1 + e^{-c\delta}(C_1 \cos(d\delta) + C_2 \sin(d\delta))]}{\delta^2(c^2 + d^2)^2}. \quad (\text{A11})$$

For the case of $b = 0$ and $c = 0$ (called the “real” term by [Foreman-Mackey et al. 2017](#)), the integrated kernel simplifies to

$$k_{FF}(\delta, \Delta) = \frac{2ae^{-c\Delta}}{(c\delta)^2} \begin{cases} e^{-c\delta} - 1 + c\delta & \Delta = 0 \\ [c(\delta - \Delta) - \sinh(c\delta - c\Delta)]e^{c\Delta} + \cosh(c\delta) - 1 & 0 < \Delta \leq \delta \\ \cosh(c\delta) - 1 & \delta < \Delta \end{cases} \quad (\text{A12})$$

In *celerite*, this exposure-averaging treatment is handled by the `TermConvolution` term.

B. SSM FOR THE STOCHASTICALLY DRIVEN DAMPED HARMONIC OSCILLATOR

In this appendix, we work out the full state-space representation of the SHO, starting from the GP kernel. While the SHO is our kernel of interest that requires exposure-integrated treatment (to model asteroseismic oscillations), it is also an excellent pedagogical choice since it has a familiar SDE that will guide us to the state-space form. The general steps are: kernel function $\rightarrow$ PSD $\rightarrow$ transfer function $\rightarrow$ SDE $\rightarrow$ state-space matrices.

The GP kernel for an SHO with parameters S_0 , ω_0 , and Q has the following form ([Foreman-Mackey et al. 2017](#)),

$$k(\Delta) = \sigma^2 \exp\left(-\frac{\omega_0}{2Q}\Delta\right) \begin{cases} 1 + \omega_0\Delta & \text{for } Q = 1/2, \\ \cos(\tau) + \frac{1}{2\eta Q} \sin(\tau) & \text{for } Q > 1/2, \\ \cosh(\tau) + \frac{1}{2\eta Q} \sinh(\tau) & \text{for } Q < 1/2, \end{cases} \quad (\text{Kernel})$$

where we have used the shorthand variables $\sigma = \sqrt{S_0\omega_0 Q}$, $\tau = \eta\omega_0\Delta$, and $\eta = \sqrt{|1 - 1/(4Q^2)|}$. Per the Wiener-Khintchine theorem, the corresponding PSD is

$$S(\omega) = \frac{2S_0\omega_0^4}{(\omega^2 - \omega_0^2)^2 + \omega_0^2\omega^2/Q^2}, \quad (\text{PSD})$$

which is clearly a rational function with all the dependence on ω in the denominator. The next step, called *minimum-phase spectral factorization* (e.g., Chapter 9 of [Anderson & Moore 1979](#)) in fields such as signal processing, is to identify the transfer function $H(i\omega)$ that satisfies Eq. 2.9 and defines a causal and stable (i.e. minimum phase) system. We can notice that the denominator of Eq. PSD can be factored like so:

$$S(\omega) = \frac{2S_0\omega_0^4}{\omega^4 - 2\omega_0^2\omega^2 + \frac{\omega_0^2}{Q^2}\omega^2 + \omega_0^4} = \frac{2S_0\omega_0^4}{\left((i\omega)^2 + \frac{\omega_0}{Q}(i\omega) + \omega_0^2\right)\left((-i\omega)^2 + \frac{\omega_0}{Q}(-i\omega) + \omega_0^2\right)}. \quad (\text{B1})$$

Once we define

$$q = 2S_0\omega_0^4 = \frac{2\omega_0^3}{Q}\sigma^2, \quad (\text{B2})$$

we are left with a complex conjugate pair of possible $H(i\omega)$ that both satisfy Eq. 2.9,

$$H_+(i\omega) = \frac{1}{(i\omega)^2 + \frac{\omega_0}{Q}(i\omega) + \omega_0^2}, \quad H_-(i\omega) = \frac{1}{(-i\omega)^2 + \frac{\omega_0}{Q}(-i\omega) + \omega_0^2}. \quad (\text{B3})$$

The minimum-phase option is identified by extending these two functions to the complex s -plane and selecting the one whose poles lie in the left half-plane, i.e., whose poles satisfy $\text{Re}(s) < 0$. That is, the appropriate transfer function for the SHO is

$$H(s) = H_+(s) = \frac{1}{s^2 + \frac{\omega_0}{Q}s + \omega_0^2}. \quad (\text{Transfer function})$$

Since the transfer function of an LTI system is, by construction, the ratio of the output to the input in the frequency domain, with the assumption that all initial conditions are zero (e.g., [Flower 1993](#), [Chen 1999](#)), we can use Eq. 2.10 to get the SDE in the Laplace domain,

$$H(s) = \frac{X(s)}{W(s)} \rightarrow s^2 X(s) + s \frac{\omega_0}{Q} X(s) + \omega_0^2 X(s) = W(s), \quad (\text{B4})$$

where $X(s)$ and $W(s)$ are the Laplace transforms of $x(t)$ and $w(t)$ (the output and the input, respectively). Inverse Laplace transforming Eq. B4, again under the assumption that all initial conditions are zero, yields the SDE for the latent state $x(t)$ of an SHO,

$$\frac{d^2 x}{dt^2} + \frac{\omega_0}{Q} \frac{dx}{dt} + \omega_0^2 x(t) = w(t), \quad (\text{SDE})$$

which we can recognize from physics. To define the SSM, we want to put this into the form of Eq. 2.5,

$$\frac{d\mathbf{x}}{dt} = \mathbf{F}\mathbf{x}(t) + \mathbf{L}w(t), \quad (\text{B5})$$

which we can do by setting $\mathbf{x}(t) = [x, \dot{x}]^T$ and reading off the coefficients to build $\mathbf{F}$ and $\mathbf{L}$ (Eq. 2.14):

$$\mathbf{F} = \begin{pmatrix} 0 & 1 \\ -\omega_0^2 & -\frac{\omega_0}{Q} \end{pmatrix}, \quad \mathbf{L} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (\text{B6})$$

To get Q_c and $\mathbf{P}_\infty$, we use these $\mathbf{F}$ and $\mathbf{L}$ and define a dummy $\mathbf{P}_\infty = \begin{pmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{pmatrix}$ and substitute into Eq. 2.15:

$$\begin{pmatrix} 0 & 1 \\ -\omega_0^2 & -\frac{\omega_0}{Q} \end{pmatrix} \begin{pmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{pmatrix} + \begin{pmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{pmatrix} \begin{pmatrix} 0 & -\omega_0^2 \\ 1 & -\frac{\omega_0}{Q} \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & Q_c \end{pmatrix} = 0.$$

Working out the multiplications yields four equations which we can solve for each element of $\mathbf{P}_\infty$,

$$\mathbf{P}_\infty = \begin{pmatrix} \frac{Q Q_c}{2\omega_0^3} & 0 \\ 0 & \frac{Q Q_c}{2\omega_0} \end{pmatrix}. \quad (\text{B7})$$

Then, setting the $P_{\infty,11}$ element (stationary variance in the state) equal to $k(\Delta = 0) = \sigma^2$ we have

$$Q_c = \frac{2\omega_0^3}{Q} \sigma^2, \quad \mathbf{P}_\infty = \sigma^2 \begin{pmatrix} 1 & 0 \\ 0 & \omega_0^2 \end{pmatrix}. \quad (\text{B8})$$

Note Q_c is exactly q in Eq. B2, as expected.

[Jordán et al. \(2021\)](#) worked out $\mathbf{A}_k$ and $\mathbf{Q}_k$ for the Matérn family and SHO. They derived full analytic expressions for $\exp(\mathbf{F}\Delta)$ and the Lyapunov integral for $\mathbf{Q}_k$ using a Laplace transform, matrix factorization, and a symbolic mathematics package. A perhaps easier way is to use the well-known solution $x(t)$ to the SDE for a given kernel and read off $\mathbf{A}_k$ from its definition

$$\begin{aligned} \mathbf{x}(t) &= \exp(\mathbf{F}t) \mathbf{x}(t=0) \\ &= \mathbf{A}_k \mathbf{x}(t=0) \\ \rightarrow \mathbf{A}_k &= \mathbf{x}(t)[\mathbf{x}(t=0)]^{-1}, \end{aligned} \quad (\text{B9})$$

and then determine $\mathbf{Q}_k$ from Eq. 2.19 using $\mathbf{A}_k$ and $\mathbf{P}_\infty$.

In any case, the result for the SHO is

$$\mathbf{A}_k = \Phi(\Delta_k) = \begin{cases} e^{-\omega_0 \Delta_k} \begin{pmatrix} 1 + \omega_0 \Delta_k & \Delta_k \\ -\omega_0^2 \Delta_k & 1 - \omega_0 \Delta_k \end{pmatrix} & \text{for } Q = 1/2 \\ e^{-\frac{\omega_0 \Delta_k}{2Q}} \begin{pmatrix} \cos \tau_k + \frac{1}{2\eta Q} \sin \tau_k & \frac{1}{\eta \omega_0} \sin \tau_k \\ \frac{-\omega_0}{\eta} \sin \tau_k & \cos \tau_k - \frac{1}{2\eta Q} \sin \tau_k \end{pmatrix} & \text{for } Q > 1/2 \\ e^{-\frac{\omega_0 \Delta_k}{2Q}} \begin{pmatrix} \cosh \tau_k + \frac{1}{2\eta Q} \sinh \tau_k & \frac{1}{\omega_0 \eta} \sinh \tau_k \\ \frac{-\omega_0}{\eta} \sinh \tau_k & \cosh \tau_k - \frac{1}{2\eta Q} \sinh \tau_k \end{pmatrix} & \text{for } Q < 1/2 \end{cases} \quad (\text{B10})$$

where we again used the shorthand $\tau_k = \eta \omega_0 \Delta_k$. We can then derive $\mathbf{Q}_k$ by substituting the above $\mathbf{A}_k$ into Eq. 2.19 given $\mathbf{P}_\infty$ (Eq. B8). We obtained (for the underdamped $Q > 1/2$)

$$\mathbf{Q}_k = \sigma^2 e^{-\frac{\omega_0 \Delta_k}{Q}} \begin{pmatrix} e^{\frac{\omega_0 \Delta_k}{Q}} - 1 - \frac{1}{2\eta Q} \sin(2\tau_k) - \frac{1}{2\eta^2 Q^2} \sin^2 \tau_k & \frac{\omega_0}{\eta^2 Q} \sin^2 \tau_k \\ \frac{\omega_0}{\eta^2 Q} \sin^2 \tau_k & \omega_0^2 \left[e^{\frac{\omega_0 \Delta_k}{Q}} - 1 + \frac{1}{2\eta Q} \sin(2\tau_k) - \frac{1}{2\eta^2 Q^2} \sin^2 \tau_k \right] \end{pmatrix}, \quad (\text{B11})$$

We note that Eq. B11 differs from that in Jordán et al. (2021) (their Eq. 26) by a factor of $Q/2\omega_0^3\sigma^2$ on the diagonals and Q/σ^2 on the off-diagonals. In fact, their expression omits S_0 entirely. We verified Eq. B11 is correct by both numerically integrating Eq. 2.19 as well as computing $\mathbf{Q}$ from a Van Loan matrix exponential (e.g. Eq. B13)—all were equivalent. Importantly, using this definition in an augmented SSM with an integral state yielded identical results to the full integrated SHO kernel of L26 (see Figure 2).

With Eq. B10 in hand, we can compute the integrated transition matrix,

$$\begin{aligned} \bar{\Phi}(\Delta) &= \int_0^\Delta \Phi(t) dt \\ &= \frac{e^{at}}{a^2 + b^2} \begin{pmatrix} a \cos \tau + b \sin \tau + A(a \sin \tau - b \cos \tau) & B(a \sin \tau - b \cos \tau) \\ C(a \sin \tau - b \cos \tau) & a \cos \tau + b \sin \tau - A(a \sin \tau - b \cos \tau) \end{pmatrix} \Bigg|_{t=0}^{t=\Delta} \end{aligned} \quad (\text{B12})$$

where here $\tau = \eta \omega_0 t$, and $A = 1/2\eta Q$, $B = 1/\eta \omega_0$, $C = -\omega_0/\eta$, $a = -\omega_0/2Q$, and $b = \eta \omega_0$.

For datasets where the numerical value of $\mathbf{F}\Delta$ is never larger than $\sim 10^5$, it is computationally fast and efficient to obtain $\tilde{\mathbf{Q}}$ from $\tilde{\mathbf{F}}$ and $\tilde{\mathbf{L}}Q_c\tilde{\mathbf{L}}^T$ via its Van Loan matrix exponential (Van Loan 1978)

$$\tilde{\mathbf{Q}}(\Delta) = \mathbf{F}_3^T \mathbf{G}_2, \quad \text{where } \exp \left[\begin{pmatrix} -\tilde{\mathbf{F}} & \tilde{\mathbf{L}}Q_c\tilde{\mathbf{L}}^T \\ \mathbf{0} & \tilde{\mathbf{F}}^T \end{pmatrix} \Delta \right] = \begin{pmatrix} \mathbf{F}_2 & \mathbf{G}_2 \\ \mathbf{0} & \mathbf{F}_3 \end{pmatrix}. \quad (\text{B13})$$

However, since we have $\mathbf{Q}$ analytically (Eq. B11), a more numerically stable means of constructing $\tilde{\mathbf{Q}}$ is to assemble it in its block form (Eq. 3.10). The top-left block is just $\mathbf{Q}$. The remaining blocks can be computed numerically from Eqs. 1.3 ($\mathbf{M}(\Delta)$) and 1.4 ($\mathbf{W}(\Delta)$) in Van Loan (1978),

$$\tilde{\mathbf{Q}}(\Delta) = \begin{pmatrix} \mathbf{Q}(\Delta) & \mathbf{M}(\Delta) \\ \mathbf{M}(\Delta)^T & \mathbf{W}(\Delta) \end{pmatrix} = \begin{pmatrix} \mathbf{Q}(\Delta) & \mathbf{F}_3^T \mathbf{H}_2 \\ (\mathbf{F}_3^T \mathbf{H}_2)^T & (\mathbf{F}_3^T \mathbf{K}_1) + (\mathbf{F}_3^T \mathbf{K}_1)^T \end{pmatrix}, \quad (\text{B14})$$

where

$$\exp \left[\begin{pmatrix} -\mathbf{F} & \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\mathbf{F} & \mathbf{L}Q_c\mathbf{L}^T & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{F}^T & \mathbf{I} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{pmatrix} \Delta \right] = \begin{pmatrix} \mathbf{F}_1 & \mathbf{G}_1 & \mathbf{H}_1 & \mathbf{K}_1 \\ \mathbf{0} & \mathbf{F}_2 & \mathbf{G}_2 & \mathbf{H}_2 \\ \mathbf{0} & \mathbf{0} & \mathbf{F}_3 & \mathbf{G}_3 \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{F}_4 \end{pmatrix}. \quad (\text{B15})$$

Recall that because we only use the first row of $\tilde{\Phi}$ in $\tilde{\Phi}$, we likewise only need the first column of $\mathbf{M}$, which we call $\tilde{Q}_{12}$, the first row of $\mathbf{M}^T$ (which is just $\tilde{Q}_{12}^T$), and the top-left element of $\mathbf{W}$, which we call $\tilde{Q}_{22}$, to give $\tilde{Q}$ the correct shape

$$\tilde{Q}(\Delta) = \begin{pmatrix} \mathbf{Q}(\Delta) & \tilde{Q}_{12}(\Delta) \\ \tilde{Q}_{12}^T(\Delta) & \tilde{Q}_{22}(\Delta) \end{pmatrix}. \quad (\text{B16})$$

In the case of the SHO, we can instead derive these analytically from

$$\tilde{Q}_{12} \equiv \text{first column of } \int_0^\Delta \Phi(t) \mathbf{L} \mathbf{Q}_c \mathbf{L}^T \bar{\Phi}(t)^T dt, \quad (\text{B17})$$

$$\tilde{Q}_{22} \equiv \text{first element of } \int_0^\Delta \bar{\Phi}(t) \mathbf{L} \mathbf{Q}_c \mathbf{L}^T \bar{\Phi}(t)^T dt. \quad (\text{B18})$$

For $Q > 1/2$, these are

$$\tilde{Q}_{12}(\Delta) = \sigma^2 \begin{pmatrix} \frac{1}{Q\omega_0} [e^{-\frac{\omega_0\Delta}{2Q}} (\cos \tau_k + A \sin \tau_k) - 1]^2 \\ A e^{-\frac{\omega_0\Delta}{2Q}} [4 \sin \tau_k - e^{-\frac{\omega_0\Delta}{2Q}} \sin(2\tau_k)] - 2A^2 e^{-\frac{\omega_0\Delta}{Q}} \sin^2(\tau_k) + e^{-\frac{\omega_0\Delta}{Q}} - 1 \end{pmatrix}, \quad (\text{B19})$$

and

$$\begin{aligned} \tilde{Q}_{22}(\Delta) = \frac{\sigma^2}{4Q^2\omega_0^2} & \left[8Q\omega_0\Delta + 4Q^2 - 12 + A^2 e^{-\frac{\omega_0\Delta}{Q}} (\cos(2\tau_k) - 16Q^4) \right. \\ & \left. + 16e^{-\frac{\omega_0\Delta}{2Q}} (\cos \tau_k + (1 - 2Q^2)A \sin \tau_k) + e^{-\frac{\omega_0\Delta}{Q}} \left(\frac{1 - 3A^2}{A} \sin(2\tau_k) - 3 \cos(2\tau_k) \right) \right], \end{aligned} \quad (\text{B20})$$

where again $\tau = \eta\omega_0\Delta$ and $A = 1/2\eta Q$. We can see from Eq. B20 that $\tilde{Q}_{22}$ grows linearly with Δ as $\Delta \rightarrow \infty$, which is the source of numerical instability when computing this term via matrix exponentials.

C. MULTICOMPONENT KERNELS

C.1. Sums of kernels

In the GP framework, a sum of covariance kernels is itself a GP kernel (Rasmussen & Williams 2006). That is,

$$\mathbf{K} = \mathbf{K}_1 + \mathbf{K}_2 + \dots + \mathbf{K}_M = \sum_{m=1}^M \mathbf{K}_m \quad (\text{C1})$$

also defines a GP. In the state-space framework, a model which is the sum of other SSMs can analogously be constructed by stacking the components in a block-diagonal form (Algorithm 12.9 in Särkkä & Solin 2019)

$$\begin{aligned} \mathbf{F} &= \text{blkdiag}(\mathbf{F}_1, \mathbf{F}_2, \dots), & \mathbf{L} &= \text{blkdiag}(\mathbf{L}_1, \mathbf{L}_2, \dots), \\ \mathbf{Q}_c &= \text{blkdiag}(\mathbf{Q}_{c,1}, \mathbf{Q}_{c,2}, \dots), & \mathbf{P}_\infty &= \text{blkdiag}(\mathbf{P}_{\infty,1}, \mathbf{P}_{\infty,2}, \dots), \end{aligned} \quad (\text{C2})$$

The matrix exponential $\Phi(\Delta) = \exp(\mathbf{F}\Delta)$ is also simply the block-diagonal matrix of component matrix exponentials, per the definition of the matrix exponential

$$\begin{aligned} \Phi(\Delta) &= \exp(\text{blkdiag}(\mathbf{F}_1, \mathbf{F}_2, \dots)\Delta) \\ &= \text{blkdiag}(\Phi_1, \Phi_2, \dots)(\Delta). \end{aligned} \quad (\text{C3})$$

Likewise the process noise also takes on a block-diagonal form:

$$\mathbf{Q} = \text{blkdiag}(\mathbf{Q}_1, \mathbf{Q}_2, \dots). \quad (\text{C4})$$

The rest of the machinery (Kalman/RTS) proceeds as before. Each component is therefore treated independently in the process dynamics but becomes coupled by the stacked measurement model $\mathbf{H} = (\mathbf{H}_1 \mathbf{H}_2 \dots)$. The overall state dimension is the sum of the component models, $d = \sum_{m=1}^M d_m$.

C.2. Products of kernels

Likewise, products of kernels also produce valid covariance kernels (Rasmussen & Williams 2006). That is,

$$\mathbf{K} = \mathbf{K}_1 \mathbf{K}_2 \dots \mathbf{K}_M = \prod_{m=1}^M \mathbf{K}_m \quad (\text{C5})$$

defines a GP. Algorithm 12.10 in (Särkkä & Solin 2019) gives the machinery for the product of SSMs,

$$\begin{aligned} \mathbf{F} &= \mathbf{F}_1 \otimes \mathbf{I} + \mathbf{I} \otimes \mathbf{F}_2 \dots, \\ \mathbf{P}_\infty &= \mathbf{P}_{\infty 1} \otimes \mathbf{P}_{\infty 2} \dots, \quad \mathbf{H} = \mathbf{H}_1 \otimes \mathbf{H}_2 \dots \end{aligned} \quad (\text{C6})$$

where $\otimes$ is the Kronecker product (which also yields block matrices). As a result, the dimension of model is now the product of the component dimensions, $d = \prod_{m=1}^M d_m$, making products much more expensive than sums of kernels in the state-space.

The matrix exponential $\mathbf{A} = \Phi(\Delta) = \exp(\mathbf{F}\Delta)$ also becomes the Kronecker product of the component matrix exponentials,

$$\begin{aligned} \Phi(\Delta) &= \exp(\mathbf{F}\Delta) = \exp(\mathbf{F}_1\Delta \otimes \mathbf{I} + \mathbf{F}_2\Delta \otimes \mathbf{I} \dots) \\ &= \Phi_1(\Delta) \otimes \Phi_2(\Delta) \dots \end{aligned} \quad (\text{C7})$$

Substituting the above $\mathbf{F}$ and $\mathbf{P}_\infty$ into the Lyapunov equation (Eq. 2.15), for a product of two kernels, we get

$$\mathbf{L}\mathbf{Q}_c\mathbf{L}^T = (\mathbf{L}_1\mathbf{Q}_{c,1}\mathbf{L}_1^T) \otimes \mathbf{P}_{\infty,2} + \mathbf{P}_{\infty,1} \otimes (\mathbf{L}_2\mathbf{Q}_{c,2}\mathbf{L}_2^T). \quad (\text{C8})$$

Any $\mathbf{L}$ and $\mathbf{Q}_c$ that satisfy this are valid; a convenient choice is $\mathbf{L} = \mathbf{I}$ and $\mathbf{Q}_c$ equal to the right-hand side of Eq. C8. Relatedly, $\mathbf{Q}$ is best determined from $\mathbf{P}_\infty$ and $\mathbf{A}$ via Eq. 2.19, which can be factored for a product of two kernels as

$$\mathbf{Q} = \mathbf{P}_{\infty,1} \otimes \mathbf{Q}_2 + \mathbf{Q}_1 \otimes \mathbf{P}_{\infty,2} - \mathbf{Q}_1 \otimes \mathbf{Q}_2. \quad (\text{C9})$$

C.3. Component-wise predictive means and variances

Say the kernel is a sum of M component kernels,

$$\mathbf{K} = \mathbf{K}_1 + \mathbf{K}_2 + \dots + \mathbf{K}_M = \sum_{m=1}^M \mathbf{K}_m. \quad (\text{C10})$$

In GP regression, we can compute the predictive mean and variance from any of the individual kernel components by substituting that component kernel into Eq. 2.4 like so

$$\begin{aligned} \boldsymbol{\mu}_{GP,m} &= \mathbf{K}_{m,*}^T (\mathbf{K} + \mathbf{N})^{-1} \mathbf{y} \\ \boldsymbol{\Sigma}_{GP,m} &= \mathbf{K}_{m,**} - \mathbf{K}_{m,*}^T (\mathbf{K} + \mathbf{N})^{-1} \mathbf{K}_{m,*}. \end{aligned} \quad (\text{C11})$$

Basically, wherever the kernel is computed with test points, we use the component kernel of interest. The overall mean and variance can be reconstructed from the components via (Duvenaud 2014)

$$\boldsymbol{\mu}_{GP} = \sum_{m=1}^M \boldsymbol{\mu}_{GP,m}, \quad \boldsymbol{\Sigma}_{GP} = \sum_{m=1}^M \boldsymbol{\Sigma}_{GP,m} - \sum_{m \neq l}^M \mathbf{K}_{m,*}^T \mathbf{K}^{-1} \mathbf{K}_{l,*}. \quad (\text{C12})$$

To do the same in the state-space framework, recall that the structure of a sum/product of kernels given in the previous sections treat the dynamics of each component as independent (i.e., the matrices are combined in block-diagonal form). Thus, the Kalman/RTS algorithms by construction provide us with all of the component means and

variances, which get summed together when projecting through the coupled observation matrix $\mathbf{H} = (\mathbf{H}_1 \ \mathbf{H}_2 \ \dots)$. To pick out just one component, we can simply do

$$\begin{aligned}\boldsymbol{\mu}_{GP,m} &= \mathbf{H}^m \hat{\mathbf{m}}, \\ \boldsymbol{\Sigma}_{GP,m} &= \mathbf{H}^m \hat{\mathbf{P}} \mathbf{H}^{mT}\end{aligned}\tag{C13}$$

where

$$\mathbf{H}^m = (0, \dots, \mathbf{H}_m, 0, \dots)\tag{C14}$$

picks out just the observation matrix for the component of interest.